

SQLAlchemy 2.1 Documentation

Release: **2.1.0b1** **BETA RELEASE** | Release Date: January 21, 2026

Search terms:

Search

[Home](#) | [Download this Documentation](#)

- **Previous:** [ORM Extensions](#)
- **Next:** [Association Proxy](#)
- **Up:** [Home](#)
 - [SQLAlchemy ORM](#)
 - [ORM Extensions](#)
- **On this page:**
 - [Asynchronous I/O \(asyncio\)](#)
 - [Asyncio Platform Installation Notes](#)
 - [Synopsis - Core](#)
 - [Synopsis - ORM](#)
 - [Using AsyncSession with Concurrent Tasks](#)
 - [Preventing Implicit IO when Using AsyncSession](#)
 - [Running Synchronous Methods and Functions under asyncio](#)
 - [Using events with the asyncio extension](#)
 - [Examples of Event Listeners with Async Engines / Sessions / Sessionmakers](#)
 - [Using awaitable-only driver methods in connection pool and other events](#)
 - [Using multiple asyncio event loops](#)
 - [Using asyncio scoped session](#)
 - [Using the Inspector to inspect schema objects](#)
 - [Engine API Documentation](#)
 - [create_async_engine\(\)](#)
 - [async_engine_from_config\(\)](#)
 - [create_async_pool_from_url\(\)](#)
 - [AsyncEngine](#)
 - [AsyncEngine.begin\(\)](#)
 - [AsyncEngine.clear_compiled_cache\(\)](#)
 - [AsyncEngine.connect\(\)](#)
 - [AsyncEngine.dialect](#)
 - [AsyncEngine.dispose\(\)](#)
 - [AsyncEngine.driver](#)
 - [AsyncEngine.echo](#)
 - [AsyncEngine.engine](#)
 - [AsyncEngine.execution_options\(\)](#)
 - [AsyncEngine.get_execution_options\(\)](#)
 - [AsyncEngine.name](#)
 - [AsyncEngine.pool](#)
 - [AsyncEngine.raw_connection\(\)](#)
 - [AsyncEngine.sync_engine](#)
 - [AsyncEngine.update_execution_options\(\)](#)
 - [AsyncEngine.url](#)
 - [AsyncConnection](#)
 - [AsyncConnection.aclose\(\)](#)
 - [AsyncConnection.begin\(\)](#)
 - [AsyncConnection.begin_nested\(\)](#)
 - [AsyncConnection.close\(\)](#)
 - [AsyncConnection.closed](#)
 - [AsyncConnection.commit\(\)](#)
 - [AsyncConnection.connection](#)
 - [AsyncConnection.default_isolation_level](#)
 - [AsyncConnection.dialect](#)
 - [AsyncConnection.exec_driver_sql\(\)](#)
 - [AsyncConnection.execute\(\)](#)

- `AsyncConnection.execution_options()`
- `AsyncConnection.get_nested_transaction()`
- `AsyncConnection.get_raw_connection()`
- `AsyncConnection.get_transaction()`
- `AsyncConnection.in_nested_transaction()`
- `AsyncConnection.in_transaction()`
- `AsyncConnection.info`
- `AsyncConnection.invalidate()`
- `AsyncConnection.invalidated`
- `AsyncConnection.rollback()`
- `AsyncConnection.run_sync()`
- `AsyncConnection.scalar()`
- `AsyncConnection.scalars()`
- `AsyncConnection.start()`
- `AsyncConnection.stream()`
- `AsyncConnection.stream_scalars()`
- `AsyncConnection.sync_connection`
- `AsyncConnection.sync_engine`
- `AsyncTransaction`
 - `AsyncTransaction.close()`
 - `AsyncTransaction.commit()`
 - `AsyncTransaction.rollback()`
 - `AsyncTransaction.start()`
- `Result Set API Documentation`
 - `AsyncResult`
 - `AsyncResult.all()`
 - `AsyncResult.close()`
 - `AsyncResult.closed`
 - `AsyncResult.columns()`
 - `AsyncResult.fetchall()`
 - `AsyncResult.fetchmany()`
 - `AsyncResult.fetchone()`
 - `AsyncResult.first()`
 - `AsyncResult.freeze()`
 - `AsyncResult.keys()`
 - `AsyncResult.mappings()`
 - `AsyncResult.one()`
 - `AsyncResult.one_or_none()`
 - `AsyncResult.partitions()`
 - `AsyncResult.scalar()`
 - `AsyncResult.scalar_one()`
 - `AsyncResult.scalar_one_or_none()`
 - `AsyncResult.scalars()`
 - `AsyncResult.t`
 - `AsyncResult.tuples()`
 - `AsyncResult.unique()`
 - `AsyncResult.yield_per()`
 - `AsyncScalarResult`
 - `AsyncScalarResult.all()`
 - `AsyncScalarResult.close()`
 - `AsyncScalarResult.closed`
 - `AsyncScalarResult.fetchall()`
 - `AsyncScalarResult.fetchmany()`
 - `AsyncScalarResult.first()`
 - `AsyncScalarResult.one()`
 - `AsyncScalarResult.one_or_none()`
 - `AsyncScalarResult.partitions()`
 - `AsyncScalarResult.unique()`
 - `AsyncScalarResult.yield_per()`
 - `AsyncMappingResult`
 - `AsyncMappingResult.all()`
 - `AsyncMappingResult.close()`

- `AsyncMappingResult.closed`
- `AsyncMappingResult.columns()`
- `AsyncMappingResult.fetchall()`
- `AsyncMappingResult.fetchmany()`
- `AsyncMappingResult.fetchone()`
- `AsyncMappingResult.first()`
- `AsyncMappingResult.keys()`
- `AsyncMappingResult.one()`
- `AsyncMappingResult.one_or_none()`
- `AsyncMappingResult.partitions()`
- `AsyncMappingResult.unique()`
- `AsyncMappingResult.yield_per()`
- `AsyncTupleResult`
- **ORM Session API Documentation**
 - `async_object_session()`
 - `async_session()`
 - `close_all_sessions()`
 - `async_sessionmaker`
 - `async_sessionmaker.__call__()`
 - `async_sessionmaker.__init__()`
 - `async_sessionmaker.begin()`
 - `async_sessionmaker.configure()`
 - `async_scoped_session`
 - `async_scoped_session.__call__()`
 - `async_scoped_session.__init__()`
 - `async_scoped_session.aclose()`
 - `async_scoped_session.add()`
 - `async_scoped_session.add_all()`
 - `async_scoped_session.autoflush`
 - `async_scoped_session.begin()`
 - `async_scoped_session.begin_nested()`
 - `async_scoped_session.bind`
 - `async_scoped_session.close()`
 - `async_scoped_session.close_all()`
 - `async_scoped_session.commit()`
 - `async_scoped_session.configure()`
 - `async_scoped_session.connection()`
 - `async_scoped_session.delete()`
 - `async_scoped_session.delete_all()`
 - `async_scoped_session.deleted`
 - `async_scoped_session.dirty`
 - `async_scoped_session.execute()`
 - `async_scoped_session.execution_options`
 - `async_scoped_session.expire()`
 - `async_scoped_session.expire_all()`
 - `async_scoped_session.expunge()`
 - `async_scoped_session.expunge_all()`
 - `async_scoped_session.flush()`
 - `async_scoped_session.get()`
 - `async_scoped_session.get_bind()`
 - `async_scoped_session.get_one()`
 - `async_scoped_session.identity_key()`
 - `async_scoped_session.identity_map`
 - `async_scoped_session.info`
 - `async_scoped_session.invalidate()`
 - `async_scoped_session.is_active`
 - `async_scoped_session.is_modified()`
 - `async_scoped_session.merge()`
 - `async_scoped_session.merge_all()`
 - `async_scoped_session.new`
 - `async_scoped_session.no_autoflush`
 - `async_scoped_session.object_session()`

- `async_scoped_session.refresh()`
- `async_scoped_session.remove()`
- `async_scoped_session.reset()`
- `async_scoped_session.rollback()`
- `async_scoped_session.scalar()`
- `async_scoped_session.scalars()`
- `async_scoped_session.session_factory`
- `async_scoped_session.stream()`
- `async_scoped_session.stream_scalars()`
- **AsyncAttrs**
 - `AsyncAttrs.awaitable_attrs`
- **AsyncSession**
 - `AsyncSession.__init__()`
 - `AsyncSession.aclose()`
 - `AsyncSession.add()`
 - `AsyncSession.add_all()`
 - `AsyncSession.autoflush`
 - `AsyncSession.begin()`
 - `AsyncSession.begin_nested()`
 - `AsyncSession.close()`
 - `AsyncSession.close_all()`
 - `AsyncSession.commit()`
 - `AsyncSession.connection()`
 - `AsyncSession.delete()`
 - `AsyncSession.delete_all()`
 - `AsyncSession.deleted`
 - `AsyncSession.dirty`
 - `AsyncSession.execute()`
 - `AsyncSession.execution_options`
 - `AsyncSession.expire()`
 - `AsyncSession.expire_all()`
 - `AsyncSession.expunge()`
 - `AsyncSession.expunge_all()`
 - `AsyncSession.flush()`
 - `AsyncSession.get()`
 - `AsyncSession.get_bind()`
 - `AsyncSession.get_nested_transaction()`
 - `AsyncSession.get_one()`
 - `AsyncSession.get_transaction()`
 - `AsyncSession.identity_key()`
 - `AsyncSession.identity_map`
 - `AsyncSession.in_nested_transaction()`
 - `AsyncSession.in_transaction()`
 - `AsyncSession.info`
 - `AsyncSession.invalidate()`
 - `AsyncSession.is_active`
 - `AsyncSession.is_modified()`
 - `AsyncSession.merge()`
 - `AsyncSession.merge_all()`
 - `AsyncSession.new`
 - `AsyncSession.no_autoflush`
 - `AsyncSession.object_session()`
 - `AsyncSession.refresh()`
 - `AsyncSession.reset()`
 - `AsyncSession.rollback()`
 - `AsyncSession.run_sync()`
 - `AsyncSession.scalar()`
 - `AsyncSession.scalars()`
 - `AsyncSession.stream()`
 - `AsyncSession.stream_scalars()`
 - `AsyncSession.sync_session`
 - `AsyncSession.sync_session_class`

- `AsyncSessionTransaction`
 - `AsyncSessionTransaction.commit()`
 - `AsyncSessionTransaction.rollback()`

Asynchronous I/O (asyncio)

Support for Python asyncio. Support for Core and ORM usage is included, using asyncio-compatible dialects.

Added in version 1.4.

Warning

Please read [Asyncio Platform Installation Notes](#) for important platform installation notes on **all** platforms.

See also

- [Asynchronous IO Support for Core and ORM](#) - initial feature announcement
- [Asyncio Integration](#) - example scripts illustrating working examples of Core and ORM use within the asyncio extension.

Asyncio Platform Installation Notes

The asyncio extension depends upon the `greenlet` library. This dependency is **not installed by default**.

To install SQLAlchemy while ensuring the `greenlet` dependency is present, the `[asyncio] setuptools extra` may be installed as follows, which will include also instruct `pip` to install `greenlet`:

```
pip install sqlalchemy[asyncio]
```

Note that installation of `greenlet` on platforms that do not have a pre-built wheel file means that `greenlet` will be built from source, which requires that Python's development libraries also be present.

Changed in version 2.1: `greenlet` is no longer installed by default; to use the asyncio extension, the `sqlalchemy[asyncio]` target must be used.

Synopsis - Core

For Core use, the `create_async_engine()` function creates an instance of `AsyncEngine` which then offers an async version of the traditional `Engine` API. The `AsyncEngine` delivers an `AsyncConnection` via its `AsyncEngine.connect()` and `AsyncEngine.begin()` methods which both deliver asynchronous context managers. The `AsyncConnection` can then invoke statements using either the `AsyncConnection.execute()` method to deliver a buffered `Result`, or the `AsyncConnection.stream()` method to deliver a streaming server-side `AsyncResult`:

```
>>> import asyncio

>>> from sqlalchemy import Column
>>> from sqlalchemy import MetaData
>>> from sqlalchemy import select
>>> from sqlalchemy import String
>>> from sqlalchemy import Table
>>> from sqlalchemy.ext.asyncio import create_async_engine

>>> meta = MetaData()
>>> t1 = Table("t1", meta, Column("name", String(50), primary_key=True))

>>> async def async_main() -> None:
...     engine = create_async_engine("sqlite+aiosqlite://", echo=True)
...
...     async with engine.begin() as conn:
...         await conn.run_sync(meta.drop_all)
...         await conn.run_sync(meta.create_all)
...
...         await conn.execute(
...             t1.insert(), [{"name": "some name 1"}, {"name": "some name 2"}]
...         )
...
...     async with engine.connect() as conn:
...         # select a Result, which will be delivered with buffered
...         # results
...         result = await conn.execute(select(t1).where(t1.c.name == "some name 1"))
...
...         print(result.fetchall())
...
...     # for AsyncEngine created in function scope, close and
```

```
...     # clean-up pooled connections
...     await engine.dispose()

>>> asyncio.run(async_main())

BEGIN (implicit)
...
CREATE TABLE t1 (
    name VARCHAR(50) NOT NULL,
    PRIMARY KEY (name)
)
...
INSERT INTO t1 (name) VALUES (?)
[...][('some name 1',), ('some name 2',)]
COMMIT
BEGIN (implicit)
SELECT t1.name
FROM t1
WHERE t1.name = ?
[...]( 'some name 1',)
[('some name 1',)]
ROLLBACK
```

Above, the `AsyncConnection.run_sync()` method may be used to invoke special DDL functions such as `MetaData.create_all()` that don't include an awaitable hook.

Tip

It's advisable to invoke the `AsyncEngine.dispose()` method using `await` when using the `AsyncEngine` object in a scope that will go out of context and be garbage collected, as illustrated in the `async_main` function in the above example. This ensures that any connections held open by the connection pool will be properly disposed within an awaitable context. Unlike when using blocking IO, SQLAlchemy cannot properly dispose of these connections within methods like `__del__` or weakref finalizers as there is no opportunity to invoke `await`. Failing to explicitly dispose of the engine when it falls out of scope may result in warnings emitted to standard out resembling the form `RuntimeError: Event loop is closed` within garbage collection.

The `AsyncConnection` also features a “streaming” API via the `AsyncConnection.stream()` method that returns an `AsyncResult` object. This result object uses a server-side cursor and provides an async/await API, such as an async iterator:

```
async with engine.connect() as conn:
    async_result = await conn.stream(select(t1))

    async for row in async_result:
        print("row: %s" % (row,))
```

Synopsis - ORM

Using `2.0 style` querying, the `AsyncSession` class provides full ORM functionality.

Within the default mode of use, special care must be taken to avoid `lazy loading` or other expired-attribute access involving ORM relationships and column attributes; the next section `Preventing Implicit IO when Using AsyncSession` details this.

Warning

A single instance of `AsyncSession` is **not safe for use in multiple, concurrent tasks**. See the sections `Using AsyncSession with Concurrent Tasks` and `Is the Session thread-safe? Is AsyncSession safe to share in concurrent tasks?` for background.

The example below illustrates a complete example including mapper and session configuration:

```
>>> from __future__ import annotations

>>> import asyncio
>>> import datetime
>>> from typing import List

>>> from sqlalchemy import ForeignKey
>>> from sqlalchemy import func
>>> from sqlalchemy import select
>>> from sqlalchemy.ext.asyncio import AsyncAttrs
>>> from sqlalchemy.ext.asyncio import async_sessionmaker
>>> from sqlalchemy.ext.asyncio import AsyncSession
>>> from sqlalchemy.ext.asyncio import create_async_engine
>>> from sqlalchemy.orm import DeclarativeBase
>>> from sqlalchemy.orm import Mapped
>>> from sqlalchemy.orm import mapped_column
>>> from sqlalchemy.orm import relationship
>>> from sqlalchemy.orm import selectinload

>>> class Base(AsyncAttrs, DeclarativeBase):
```

```

...     pass

>>> class B(Base):
...     __tablename__ = "b"
...
...     id: Mapped[int] = mapped_column(primary_key=True)
...     a_id: Mapped[int] = mapped_column(ForeignKey("a.id"))
...     data: Mapped[str]

>>> class A(Base):
...     __tablename__ = "a"
...
...     id: Mapped[int] = mapped_column(primary_key=True)
...     data: Mapped[str]
...     create_date: Mapped[datetime.datetime] = mapped_column(server_default=func.now())
...     bs: Mapped[List[B]] = relationship()

>>> async def insert_objects(async_session: async_sessionmaker[AsyncSession]) -> None:
...     async with async_session() as session:
...         async with session.begin():
...             session.add_all(
...                 [
...                     A(bs=[B(data="b1"), B(data="b2")], data="a1"),
...                     A(bs=[], data="a2"),
...                     A(bs=[B(data="b3"), B(data="b4")], data="a3"),
...                 ]
...             )

>>> async def select_and_update_objects(
...     async_session: async_sessionmaker[AsyncSession],
... ) -> None:
...     async with async_session() as session:
...         stmt = select(A).order_by(A.id).options(selectinload(A.bs))
...
...         result = await session.execute(stmt)
...
...         for a in result.scalars():
...             print(a, a.data)
...             print(f"created at: {a.create_date}")
...             for b in a.bs:
...                 print(b, b.data)
...
...         result = await session.execute(select(A).order_by(A.id).limit(1))
...
...         a1 = result.scalars().one()
...
...         a1.data = "new data"
...
...         await session.commit()
...
...         # access attribute subsequent to commit; this is what
...         # expire_on_commit=False allows
...         print(a1.data)
...
...         # alternatively, AsyncAttrs may be used to access any attribute
...         # as an awaitable (new in 2.0.13)
...         for b1 in await a1.awaitable_attrs.bs:
...             print(b1, b1.data)

>>> async def async_main() -> None:
...     engine = create_async_engine("sqlite+aiosqlite://", echo=True)
...
...     # async_sessionmaker: a factory for new AsyncSession objects.
...     # expire_on_commit - don't expire objects after transaction commit
...     async_session = async_sessionmaker(engine, expire_on_commit=False)
...
...     async with engine.begin() as conn:
...         await conn.run_sync(Base.metadata.create_all)
...
...     await insert_objects(async_session)
...     await select_and_update_objects(async_session)
...
...     # for AsyncEngine created in function scope, close and
...     # clean-up pooled connections
...     await engine.dispose()

>>> asyncio.run(async_main())

```

```

BEGIN (implicit)
...
CREATE TABLE a (
    id INTEGER NOT NULL,
    data VARCHAR NOT NULL,
    create_date DATETIME DEFAULT CURRENT_TIMESTAMP NOT NULL,
    PRIMARY KEY (id)
)
...

```

```
CREATE TABLE b (  
    id INTEGER NOT NULL,  
    a_id INTEGER NOT NULL,  
    data VARCHAR NOT NULL,  
    PRIMARY KEY (id),  
    FOREIGN KEY(a_id) REFERENCES a (id)  
)  
...  
COMMIT  
BEGIN (implicit)  
INSERT INTO a (data) VALUES (?) RETURNING id, create_date  
[...] ('a1',)  
...  
INSERT INTO b (a_id, data) VALUES (?, ?) RETURNING id  
[...] (1, 'b2')  
...  
COMMIT  
BEGIN (implicit)  
SELECT a.id, a.data, a.create_date  
FROM a ORDER BY a.id  
[...] ()  
SELECT b.a_id, b.id, b.data  
FROM b  
WHERE b.a_id IN (?, ?, ?)  
[...] (1, 2, 3)  
<A object at ...> a1  
created at: ...  
<B object at ...> b1  
<B object at ...> b2  
<A object at ...> a2  
created at: ...  
<A object at ...> a3  
created at: ...  
<B object at ...> b3  
<B object at ...> b4  
SELECT a.id, a.data, a.create_date  
FROM a ORDER BY a.id  
LIMIT ? OFFSET ?  
[...] (1, 0)  
UPDATE a SET data=? WHERE a.id = ?  
[...] ('new data', 1)  
COMMIT  
new data  
<B object at ...> b1  
<B object at ...> b2
```

Annotated Example

In the example above, the **AsyncSession** is instantiated using the optional **async_sessionmaker** helper, which provides a factory for new **AsyncSession** objects with a fixed set of parameters, which here includes associating it with an **AsyncEngine** against particular database URL. It is then passed to other methods where it may be used in a Python asynchronous context manager (i.e. `async with:` statement) so that it is automatically closed at the end of the block; this is equivalent to calling the **AsyncSession.close()** method.

Using AsyncSession with Concurrent Tasks

The **AsyncSession** object is a **mutable, stateful object** which represents a **single, stateful database transaction in progress**. Using concurrent tasks with `asyncio`, with APIs such as `asyncio.gather()` for example, should use a **separate AsyncSession per individual task**.

See the section **Is the Session thread-safe? Is AsyncSession safe to share in concurrent tasks?** for a general description of the **Session** and **AsyncSession** with regards to how they should be used with concurrent workloads.

Preventing Implicit IO when Using AsyncSession

Using traditional `asyncio`, the application needs to avoid any points at which IO-on-attribute access may occur. Techniques that can be used to help this are below, many of which are illustrated in the preceding example.

- Attributes that are lazy-loading relationships, deferred columns or expressions, or are being accessed in expiration scenarios can take advantage of the **AsyncAttrs** mixin. This mixin, when added to a specific class or more generally to the Declarative **Base** superclass, provides an accessor **AsyncAttrs.awaitable_attrs** which delivers any attribute as an awaitable:

```
from __future__ import annotations  
  
from typing import List  
  
from sqlalchemy.ext.asyncio import AsyncAttrs  
from sqlalchemy.orm import DeclarativeBase  
from sqlalchemy.orm import Mapped  
from sqlalchemy.orm import relationship  
  
class Base(AsyncAttrs, DeclarativeBase):  
    pass  
  
class A(Base):  
    __tablename__ = "a"  
  
    # ... rest of mapping ...  
  
    bs: Mapped[List[B]] = relationship()
```



```
class B(Base):
    __tablename__ = "b"

    # ... rest of mapping ...
```

Annotated Example

Accessing the `A.bs` collection on newly loaded instances of `A` when eager loading is not in use will normally use **lazy loading**, which in order to succeed will usually emit IO to the database, which will fail under `asyncio` as no implicit IO is allowed. To access this attribute directly under `asyncio` without any prior loading operations, the attribute can be accessed as an awaitable by indicating the `AsyncAttrs.awaitable_attrs` prefix:

```
a1 = (await session.scalars(select(A))).one()
for b1 in await a1.awaitable_attrs.bs:
    print(b1)
```

The `AsyncAttrs` mixin provides a succinct facade over the internal approach that's also used by the `AsyncSession.run_sync()` method.

Added in version 2.0.13.

See also

AsyncAttrs

- Collections can be replaced with **write only collections** that will never emit IO implicitly, by using the **Write Only Relationships** feature in SQLAlchemy 2.0. Using this feature, collections are never read from, only queried using explicit SQL calls. See the example `async_orm_writeonly.py` in the **Asyncio Integration** section for an example of write-only collections used with `asyncio`.

When using write only collections, the program's behavior is simple and easy to predict regarding collections. However, the downside is that there is not any built-in system for loading many of these collections all at once, which instead would need to be performed manually. Therefore, many of the bullets below address specific techniques when using traditional lazy-loaded relationships with `asyncio`, which requires more care.

- If not using `AsyncAttrs`, relationships can be declared with `lazy="raise"` so that by default they will not attempt to emit SQL. In order to load collections, **eager loading** would be used instead.
- The most useful eager loading strategy is the `selectinload()` eager loader, which is employed in the previous example in order to eagerly load the `A.bs` collection within the scope of the `await session.execute()` call:

```
stmt = select(A).options(selectinload(A.bs))
```

- When constructing new objects, **collections are always assigned a default, empty collection**, such as a list in the above example:

```
A(bs=[], data="a2")
```

This allows the `.bs` collection on the above `A` object to be present and readable when the `A` object is flushed; otherwise, when the `A` is flushed, `.bs` would be unloaded and would raise an error on access.

- The `AsyncSession` is configured using `Session.expire_on_commit` set to `False`, so that we may access attributes on an object subsequent to a call to `AsyncSession.commit()`, as in the line at the end where we access an attribute:

```
# create AsyncSession with expire_on_commit=False
async_session = AsyncSession(engine, expire_on_commit=False)

# sessionmaker version
async_session = async_sessionmaker(engine, expire_on_commit=False)

async with async_session() as session:
    result = await session.execute(select(A).order_by(A.id))

    a1 = result.scalars().first()

    # commit would normally expire all attributes
    await session.commit()

    # access attribute subsequent to commit; this is what
    # expire_on_commit=False allows
    print(a1.data)
```

Other guidelines include:

- Methods like `AsyncSession.expire()` should be avoided in favor of `AsyncSession.refresh()`; if expiration is absolutely needed. Expiration should generally **not** be needed as `Session.expire_on_commit` should normally be set to `False` when using `asyncio`.
- A lazy-loaded relationship **can be loaded explicitly under asyncio** using `AsyncSession.refresh()`, if the desired attribute name is passed explicitly to `Session.refresh.attribute_names`, e.g.:


```
# assume a_obj is an A that has lazy loaded A.bs collection
a_obj = await async_session.get(A, [1])

# force the collection to load by naming it in attribute_names
await async_session.refresh(a_obj, ["bs"])

# collection is present
print(f"bs collection: {a_obj.bs}")
```

It’s of course preferable to use eager loading up front in order to have collections already set up without the need to lazy-load.

Added in version 2.0.4: Added support for `AsyncSession.refresh()` and the underlying `Session.refresh()` method to force lazy-loaded relationships to load, if they are named explicitly in the `Session.refresh.attribute_names` parameter. In previous versions, the relationship would be silently skipped even if named in the parameter.

- Avoid using the `all` cascade option documented at [Cascades](#) in favor of listing out the desired cascade features explicitly. The `all` cascade option implies among others the `refresh-expire` setting, which means that the `AsyncSession.refresh()` method will expire the attributes on related objects, but not necessarily refresh those related objects assuming eager loading is not configured within the `relationship()`, leaving them in an expired state.
- Appropriate loader options should be employed for `deferred()` columns, if used at all, in addition to that of `relationship()` constructs as noted above. See [Limiting which Columns Load with Column Deferral](#) for background on deferred column loading.
- The “dynamic” relationship loader strategy described at [Dynamic Relationship Loaders](#) is not compatible by default with the `asyncio` approach. It can be used directly only if invoked within the `AsyncSession.run_sync()` method described at [Running Synchronous Methods and Functions under asyncio](#), or by using its `.statement` attribute to obtain a normal select:

```
user = await session.get(User, 42)
addresses = (await session.scalars(user.addresses.statement)).all()
stmt = user.addresses.statement.where(Address.email_address.startswith("patrick"))
addresses_filter = (await session.scalars(stmt)).all()
```

The `write only` technique, introduced in version 2.0 of SQLAlchemy, is fully compatible with `asyncio` and should be preferred.

See also

“Dynamic” relationship loaders superseded by “Write Only” - notes on migration to 2.0 style

- If using `asyncio` with a database that does not support RETURNING, such as MySQL 8, server default values such as generated timestamps will not be available on newly flushed objects unless the `Mapper.eager_defaults` option is used. In SQLAlchemy 2.0, this behavior is applied automatically to backends like PostgreSQL, SQLite and MariaDB which use RETURNING to fetch new values when rows are INSERTed.

Running Synchronous Methods and Functions under asyncio

Deep Alchemy

This approach is essentially exposing publicly the mechanism by which SQLAlchemy is able to provide the `asyncio` interface in the first place. While there is no technical issue with doing so, overall the approach can probably be considered “controversial” as it works against some of the central philosophies of the `asyncio` programming model, which is essentially that any programming statement that can potentially result in IO being invoked **must** have an `await` call, lest the program does not make it explicitly clear every line at which IO may occur. This approach does not change that general idea, except that it allows a series of synchronous IO instructions to be exempted from this rule within the scope of a function call, essentially bundled up into a single awaitable.

As an alternative means of integrating traditional SQLAlchemy “lazy loading” within an `asyncio` event loop, an **optional** method known as `AsyncSession.run_sync()` is provided which will run any Python function inside of a greenlet, where traditional synchronous programming concepts will be translated to use `await` when they reach the database driver. A hypothetical approach here is an `asyncio`-oriented application can package up database-related methods into functions that are invoked using `AsyncSession.run_sync()`.

Altering the above example, if we didn’t use `selectinload()` for the `A.bs` collection, we could accomplish our treatment of these attribute accesses within a separate function:

```
import asyncio

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine

def fetch_and_update_objects(session):
    """run traditional sync-style ORM code in a function that will be
    invoked within an awaitable.

    """

    # the session object here is a traditional ORM Session.
    # all features are available here including legacy Query use.

    stmt = select(A)

    result = session.execute(stmt)
    for a1 in result.scalars():
        print(a1)

    # Lazy Loads
    for b1 in a1.bs:
```

```
print(b1)

# Legacy Query use
a1 = session.query(A).order_by(A.id).first()

a1.data = "new data"

async def async_main():
    engine = create_async_engine(
        "postgresql+asyncpg://scott:tiger@localhost/test",
        echo=True,
    )
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)
        await conn.run_sync(Base.metadata.create_all)

    async with AsyncSession(engine) as session:
        async with session.begin():
            session.add_all(
                [
                    A(bs=[B(), B()], data="a1"),
                    A(bs=[B()], data="a2"),
                    A(bs=[B(), B()], data="a3"),
                ]
            )

            await session.run_sync(fetch_and_update_objects)

            await session.commit()

# for AsyncEngine created in function scope, close and
# clean-up pooled connections
await engine.dispose()

asyncio.run(async_main())
```

The above approach of running certain functions within a “sync” runner has some parallels to an application that runs a SQLAlchemy application on top of an event-based programming library such as `gevent`. The differences are as follows:

1. unlike when using `gevent`, we can continue to use the standard Python asyncio event loop, or any custom event loop, without the need to integrate into the `gevent` event loop.
2. There is no “monkeypatching” whatsoever. The above example makes use of a real asyncio driver and the underlying SQLAlchemy connection pool is also using the Python built-in `asyncio.Queue` for pooling connections.
3. The program can freely switch between async/await code and contained functions that use sync code with virtually no performance penalty. There is no “thread executor” or any additional waiters or synchronization in use.
4. The underlying network drivers are also using pure Python asyncio concepts, no third party networking libraries as `gevent` and `eventlet` provides are in use.

Using events with the asyncio extension

The SQLAlchemy `event system` is not directly exposed by the asyncio extension, meaning there is not yet an “async” version of a SQLAlchemy event handler.

However, as the asyncio extension surrounds the usual synchronous SQLAlchemy API, regular “synchronous” style event handlers are freely available as they would be if asyncio were not used.

As detailed below, there are two current strategies to register events given asyncio-facing APIs:

- Events can be registered at the instance level (e.g. a specific `AsyncEngine` instance) by associating the event with the `sync` attribute that refers to the proxied object. For example to register the `PoolEvents.connect()` event against an `AsyncEngine` instance, use its `AsyncEngine.sync_engine` attribute as target. Targets include:

```
AsyncEngine.sync_engine

AsyncConnection.sync_connection

AsyncConnection.sync_engine

AsyncSession.sync_session
```

- To register an event at the class level, targeting all instances of the same type (e.g. all `AsyncSession` instances), use the corresponding sync-style class. For example to register the `SessionEvents.before_commit()` event against the `AsyncSession` class, use the `Session` class as the target.
- To register at the `sessionmaker` level, combine an explicit `sessionmaker` with an `async_sessionmaker` using `async_sessionmaker.sync_session_class`, and associate events with the `sessionmaker`.

When working within an event handler that is within an asyncio context, objects like the `Connection` continue to work in their usual “synchronous” way without requiring `await` or `async` usage; when messages are ultimately received by the asyncio database adapter, the calling style is transparently adapted back into the asyncio calling style. For events that are passed a DBAPI level connection, such as `PoolEvents.connect()`, the object is a `pep-249` compliant “connection” object which will adapt sync-style calls into the asyncio driver.

Examples of Event Listeners with Async Engines / Sessions / Sessionmakers

Some examples of sync style event handlers associated with async-facing API constructs are illustrated below:

- **Core Events on AsyncEngine**

In this example, we access the `AsyncEngine.sync_engine` attribute of `AsyncEngine` as the target for `ConnectionEvents` and `PoolEvents`:

```
import asyncio

from sqlalchemy import event
from sqlalchemy import text
from sqlalchemy.engine import Engine
from sqlalchemy.ext.asyncio import create_async_engine

engine = create_async_engine("postgresql+asyncpg://scott:tiger@localhost:5432/test")

# connect event on instance of Engine
@event.listens_for(engine.sync_engine, "connect")
def my_on_connect(dbapi_con, connection_record):
    print("New DBAPI connection:", dbapi_con)
    cursor = dbapi_con.cursor()

    # sync style API use for adapted DBAPI connection / cursor
    cursor.execute("select 'execute from event'")
    print(cursor.fetchone()[0])

# before_execute event on all Engine instances
@event.listens_for(Engine, "before_execute")
def my_before_execute(
    conn,
    clauseelement,
    multiparams,
    params,
    execution_options,
):
    print("before execute!")

async def go():
    async with engine.connect() as conn:
        await conn.execute(text("select 1"))
    await engine.dispose()

asyncio.run(go())
```

Output:

```
New DBAPI connection: <AdaptedConnection <asyncpg.connection.Connection object at 0x7f33f9b16960>>
execute from event
before execute!
```

- **ORM Events on AsyncSession**

In this example, we access `AsyncSession.sync_session` as the target for `SessionEvents`:

```
import asyncio

from sqlalchemy import event
from sqlalchemy import text
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.ext.asyncio import create_async_engine
from sqlalchemy.orm import Session

engine = create_async_engine("postgresql+asyncpg://scott:tiger@localhost:5432/test")

session = AsyncSession(engine)

# before_commit event on instance of Session
@event.listens_for(session.sync_session, "before_commit")
def my_before_commit(session):
    print("before commit!")

# sync style API use on Session
connection = session.connection()

# sync style API use on Connection
result = connection.execute(text("select 'execute from event'"))
print(result.first())

# after_commit event on all Session instances
@event.listens_for(Session, "after_commit")
def my_after_commit(session):
    print("after commit!")

async def go():
```



```
        await session.execute(text("select 1"))
        await session.commit()

    await session.close()
    await engine.dispose()

asyncio.run(go())
```

Output:

```
before commit!
execute from event
after commit!
```

• ORM Events on `async_sessionmaker`

For this use case, we make a `sessionmaker` as the event target, then assign it to the `async_sessionmaker` using the `async_sessionmaker.sync_session_class` parameter:

```
import asyncio

from sqlalchemy import event
from sqlalchemy.ext.asyncio import async_sessionmaker
from sqlalchemy.orm import sessionmaker

sync_maker = sessionmaker()
maker = async_sessionmaker(sync_session_class=sync_maker)

@event.listens_for(sync_maker, "before_commit")
def before_commit(session):
    print("before commit")

async def main():
    async_session = maker()

    await async_session.commit()

asyncio.run(main())
```

Output:

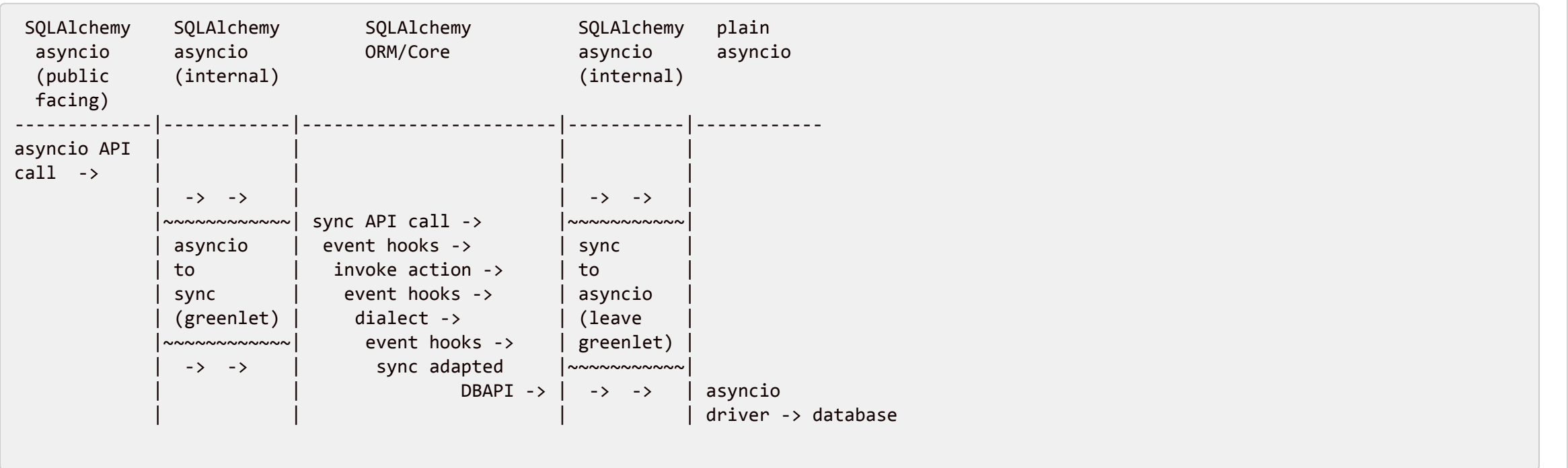
```
before commit
```

asyncio and events, two opposites

SQLAlchemy events by their nature take place within the **interior** of a particular SQLAlchemy process; that is, an event always occurs *after* some particular SQLAlchemy API has been invoked by end-user code, and *before* some other internal aspect of that API occurs.

Contrast this to the architecture of the asyncio extension, which takes place on the **exterior** of SQLAlchemy’s usual flow from end-user API to DBAPI function.

The flow of messaging may be visualized as follows:



Where above, an API call always starts as asyncio, flows through the synchronous API, and ends as asyncio, before results are propagated through this same chain in the opposite direction. In between, the message is adapted first into sync-style API use, and then back out to async style. Event hooks then by their nature occur in the middle of the “sync-style API use”. From this it follows that the API presented within event hooks occurs inside the process by which asyncio API requests have been adapted to sync, and outgoing messages to the database API will be converted to asyncio transparently.

Using awaitable-only driver methods in connection pool and other events

As discussed in the above section, event handlers such as those oriented around the **PoolEvents** event handlers receive a sync-style “DBAPI” connection, which is a wrapper object supplied by SQLAlchemy asyncio dialects to adapt the underlying asyncio “driver” connection into one that can be used by SQLAlchemy’s internals. A special use case arises when the user-defined implementation for such an event handler needs to make use of the ultimate “driver” connection directly, using awaitable only methods on that driver connection. One such example is the `.set_type_codec()` method supplied by the `asyncpg` driver.

To accommodate this use case, SQLAlchemy’s **AdaptedConnection** class provides a method **AdaptedConnection.run_async()** that allows an awaitable function to be invoked within the “synchronous” context of an event handler or other SQLAlchemy internal. This method is directly analogous to the **AsyncConnection.run_sync()** method that allows a sync-style method to run under `async`.

AdaptedConnection.run_async() should be passed a function that will accept the innermost “driver” connection as a single argument, and return an awaitable that will be invoked by the **AdaptedConnection.run_async()** method. The given function itself does not need to be declared as `async`; it’s perfectly fine for it to be a Python `lambda`, as the return awaitable value will be invoked after being returned:

```
from sqlalchemy import event
from sqlalchemy.ext.asyncio import create_async_engine

engine = create_async_engine(...)

@event.listens_for(engine.sync_engine, "connect")
def register_custom_types(dbapi_connection, *args):
    dbapi_connection.run_async(
        lambda connection: connection.set_type_codec(
            "MyCustomType",
            encoder,
            decoder, # ...
        )
    )
```

Above, the object passed to the `register_custom_types` event handler is an instance of **AdaptedConnection**, which provides a DBAPI-like interface to an underlying `async-only` driver-level connection object. The **AdaptedConnection.run_async()** method then provides access to an awaitable environment where the underlying driver level connection may be acted upon.

Added in version 1.4.30.

Using multiple asyncio event loops

An application that makes use of multiple event loops, for example in the uncommon case of combining `asyncio` with `multithreading`, should not share the same **AsyncEngine** with different event loops when using the default pool implementation.

If an **AsyncEngine** is be passed from one event loop to another, the method **AsyncEngine.dispose()** should be called before it’s reused on a new event loop. Failing to do so may lead to a `RuntimeError` along the lines of `Task <Task pending ...> got Future attached to a different loop`

If the same engine must be shared between different loop, it should be configured to disable pooling using **NullPool**, preventing the Engine from using any connection more than once:

```
from sqlalchemy.ext.asyncio import create_async_engine
from sqlalchemy.pool import NullPool

engine = create_async_engine(
    "postgresql+asyncpg://user:pass@host/dbname",
    poolclass=NullPool,
)
```

Using asyncio scoped session

The “scoped session” pattern used in threaded SQLAlchemy with the **scoped_session** object is also available in `asyncio`, using an adapted version called **async_scoped_session**.

Tip

SQLAlchemy generally does not recommend the “scoped” pattern for new development as it relies upon mutable global state that must also be explicitly torn down when work within the thread or task is complete. Particularly when using `asyncio`, it’s likely a better idea to pass the **AsyncSession** directly to the awaitable functions that need it.

When using **async_scoped_session**, as there’s no “thread-local” concept in the `asyncio` context, the “scopefunc” parameter must be provided to the constructor. The example below illustrates using the `asyncio.current_task()` function for this purpose:

```
from asyncio import current_task

from sqlalchemy.ext.asyncio import (
    async_scoped_session,
    async_sessionmaker,
)

async_session_factory = async_sessionmaker(
    some_async_engine,
    expire_on_commit=False,
)

AsyncScopedSession= async_scoped_session(
```

```
    async_session_factory,
    scopefunc=current_task,
)
some_async_session = AsyncScopedSession()
```

Warning

The “scopefunc” used by **async_scoped_session** is invoked **an arbitrary number of times** within a task, once for each time the underlying **AsyncSession** is accessed. The function should therefore be **idempotent** and lightweight, and should not attempt to create or mutate any state, such as establishing callbacks, etc.

Warning

Using `current_task()` for the “key” in the scope requires that the **async_scoped_session.remove()** method is called from within the outermost awaitable, to ensure the key is removed from the registry when the task is complete, otherwise the task handle as well as the **AsyncSession** will remain in memory, essentially creating a memory leak. See the following example which illustrates the correct use of **async_scoped_session.remove()**.

async_scoped_session includes **proxy behavior** similar to that of **scoped_session**, which means it can be treated as a **AsyncSession** directly, keeping in mind that the usual `await` keywords are necessary, including for the **async_scoped_session.remove()** method:

```
async def some_function(some_async_session, some_object):
    # use the AsyncSession directly
    some_async_session.add(some_object)

    # use the AsyncSession via the context-local proxy
    await AsyncScopedSession.commit()

    # "remove" the current proxied AsyncSession for the local context
    await AsyncScopedSession.remove()
```

Added in version 1.4.19.

Using the Inspector to inspect schema objects

SQLAlchemy does not yet offer an asyncio version of the **Inspector** (introduced at [Fine Grained Reflection with Inspector](#)), however the existing interface may be used in an asyncio context by leveraging the **AsyncConnection.run_sync()** method of **AsyncConnection**:

```
import asyncio

from sqlalchemy import inspect
from sqlalchemy.ext.asyncio import create_async_engine

engine = create_async_engine("postgresql+asyncpg://scott:tiger@localhost/test")

def use_inspector(conn):
    inspector = inspect(conn)
    # use the inspector
    print(inspector.get_view_names())
    # return any value to the caller
    return inspector.get_table_names()

async def async_main():
    async with engine.connect() as conn:
        tables = await conn.run_sync(use_inspector)

asyncio.run(async_main())
```

See also

- Reflecting Database Objects
- Runtime Inspection API

Engine API Documentation

Object Name	Description
async_engine_from_config (configuration[, prefix], **kwargs)	Create a new AsyncEngine instance using a configuration dictionary.

AsyncConnection	An asyncio proxy for a Connection .
AsyncEngine	An asyncio proxy for a Engine .
AsyncTransaction	An asyncio proxy for a Transaction .
create_async_engine (url, **kw)	Create a new async engine instance.
create_async_pool_from_url (url, **kwargs)	Create a new async engine instance.

function sqlalchemy.ext.asyncio.**create_async_engine**(url:str/*URL*, **kw:Any) → AsyncEngine

Create a new async engine instance.

Arguments passed to **create_async_engine()** are mostly identical to those passed to the **create_engine()** function. The specified dialect must be an asyncio-compatible dialect such as **asyncpg**.

Added in version 1.4.

Parameters:

async_creator –
an async callable which returns a driver-level asyncio connection. If given, the function should take no arguments, and return a new asyncio connection from the underlying asyncio database driver; the connection will be wrapped in the appropriate structures to be used with the **AsyncEngine**. Note that the parameters specified in the URL are not applied here, and the creator function should use its own connection parameters.

This parameter is the asyncio equivalent of the **create_engine.creator** parameter of the **create_engine()** function.

Added in version 2.0.16.

function sqlalchemy.ext.asyncio.**async_engine_from_config**(configuration:Dict[str, Any], prefix:str = 'sqlalchemy.', **kwargs:Any) → AsyncEngine

Create a new AsyncEngine instance using a configuration dictionary.

This function is analogous to the **engine_from_config()** function in SQLAlchemy Core, except that the requested dialect must be an asyncio-compatible dialect such as **asyncpg**. The argument signature of the function is identical to that of **engine_from_config()**.

Added in version 1.4.29.

function sqlalchemy.ext.asyncio.**create_async_pool_from_url**(url:str/*URL*, **kwargs:Any) → Pool

Create a new async engine instance.

Arguments passed to **create_async_pool_from_url()** are mostly identical to those passed to the **create_pool_from_url()** function. The specified dialect must be an asyncio-compatible dialect such as **asyncpg**.

Added in version 2.0.10.

class sqlalchemy.ext.asyncio.**AsyncEngine**

inherits from sqlalchemy.ext.asyncio.base.ProxyComparable, sqlalchemy.ext.asyncio.AsyncConnectable

An asyncio proxy for a **Engine**.

AsyncEngine is acquired using the **create_async_engine()** function:

```
from sqlalchemy.ext.asyncio import create_async_engine

engine = create_async_engine("postgresql+asyncpg://user:pass@host/dbname")
```

Added in version 1.4.

Member Name	Description
begin()	Return a context manager which when entered will deliver an AsyncConnection with an AsyncTransaction established.
clear_compiled_cache()	Clear the compiled cache associated with the dialect.
connect()	Return an AsyncConnection object.

<code>dispose()</code>	Dispose of the connection pool used by this AsyncEngine .
<code>execution_options()</code>	Return a new AsyncEngine that will provide AsyncConnection objects with the given execution options.
<code>get_execution_options()</code>	Get the non-SQL options which will take effect during execution.
<code>raw_connection()</code>	Return a “raw” DBAPI connection from the connection pool.
<code>sync_engine</code>	Reference to the sync-style Engine this AsyncEngine proxies requests towards.
<code>update_execution_options()</code>	Update the default execution_options dictionary of this Engine .

method `sqlalchemy.ext.asyncio.AsyncEngine.begin()` → AsyncIterator[**AsyncConnection**]

Return a context manager which when entered will deliver an **AsyncConnection** with an **AsyncTransaction** established.

E.g.:

```
async with async_engine.begin() as conn:
    await conn.execute(
        text("insert into table (x, y, z) values (1, 2, 3)")
    )
    await conn.execute(text("my_special_procedure(5)"))
```

method `sqlalchemy.ext.asyncio.AsyncEngine.clear_compiled_cache()` → None

Clear the compiled cache associated with the dialect.

Proxied for the **Engine** class on behalf of the **AsyncEngine** class.

This applies **only** to the built-in cache that is established via the `create_engine.query_cache_size` parameter. It will not impact any dictionary caches that were passed via the `Connection.execution_options.compiled_cache` parameter.

Added in version 1.4.

method `sqlalchemy.ext.asyncio.AsyncEngine.connect()` → **AsyncConnection**

Return an **AsyncConnection** object.

The **AsyncConnection** will procure a database connection from the underlying connection pool when it is entered as an async context manager:

```
async with async_engine.connect() as conn:
    result = await conn.execute(select(user_table))
```

The **AsyncConnection** may also be started outside of a context manager by invoking its **AsyncConnection.start()** method.

property **dialect**: **Dialect**

Proxy for the `Engine.dialect` attribute on behalf of the **AsyncEngine** class.

method `sqlalchemy.ext.asyncio.AsyncEngine.async dispose(close: bool = True)` → None

Dispose of the connection pool used by this **AsyncEngine**.

Parameters:

close –

if left at its default of `True`, has the effect of fully closing all **currently checked in** database connections. Connections that are still checked out will **not** be closed, however they will no longer be associated with this **Engine**, so when they are closed individually, eventually the **Pool** which they are associated with will be garbage collected and they will be closed out fully, if not already closed on checkin.

If set to `False`, the previous connection pool is de-referenced, and otherwise not touched in any way.

See also

Engine.dispose()

property **driver**: **Any**

Driver name of the **Dialect** in use by this **Engine**.

Proxied for the **Engine** class on behalf of the **AsyncEngine** class.

property **echo**: Any

When `True`, enable log output for this element.

Proxied for the **Engine** class on behalf of the **AsyncEngine** class.

This has the effect of setting the Python logging level for the namespace of this element’s class and object reference. A value of boolean `True` indicates that the loglevel `logging.INFO` will be set for the logger, whereas the string value `debug` will set the loglevel to `logging.DEBUG`.

property **engine**: Any

Returns this **Engine**.

Proxied for the **Engine** class on behalf of the **AsyncEngine** class.

Used for legacy schemes that accept **Connection** / **Engine** objects within the same variable.

method **sqlalchemy.ext.asyncio.AsyncEngine.execution_options**(***opt: Any*) → AsyncEngine

Return a new **AsyncEngine** that will provide **AsyncConnection** objects with the given execution options.

Proxied from **Engine.execution_options()**. See that method for details.

method **sqlalchemy.ext.asyncio.AsyncEngine.get_execution_options**() → _ExecuteOptions

Get the non-SQL options which will take effect during execution.

Proxied for the **Engine** class on behalf of the **AsyncEngine** class.

See also

Engine.execution_options()

property **name**: Any

String name of the **Dialect** in use by this **Engine**.

Proxied for the **Engine** class on behalf of the **AsyncEngine** class.

property **pool**: Pool

Proxy for the `Engine.pool` attribute on behalf of the **AsyncEngine** class.

method **sqlalchemy.ext.asyncio.AsyncEngine.async_raw_connection**() → PoolProxiedConnection

Return a “raw” DBAPI connection from the connection pool.

See also

Working with Driver SQL and Raw DBAPI Connections

attribute **sqlalchemy.ext.asyncio.AsyncEngine.sync_engine**: Engine

Reference to the sync-style **Engine** this **AsyncEngine** proxies requests towards.

This instance can be used as an event target.

See also

Using events with the asyncio extension

method **sqlalchemy.ext.asyncio.AsyncEngine.update_execution_options**(***opt: Any*) → None

Update the default execution_options dictionary of this **Engine**.

Proxied for the **Engine** class on behalf of the **AsyncEngine** class.

The given keys/values in `**opt` are added to the default execution options that will be used for all connections. The initial contents of this dictionary can be sent via the `execution_options` parameter to **create_engine()**.

See also

`Connection.execution_options()`

`Engine.execution_options()`

property `url`: URL

Proxy for the `Engine.url` attribute on behalf of the `AsyncEngine` class.

class `sqlalchemy.ext.asyncio.AsyncConnection`

inherits from `sqlalchemy.ext.asyncio.base.ProxyComparable`, `sqlalchemy.ext.asyncio.base.StartableContext`, `sqlalchemy.ext.asyncio.AsyncConnectable`

An `asyncio` proxy for a `Connection`.

`AsyncConnection` is acquired using the `AsyncEngine.connect()` method of `AsyncEngine`:

```
from sqlalchemy.ext.asyncio import create_async_engine

engine = create_async_engine("postgresql+asyncpg://user:pass@host/dbname")

async with engine.connect() as conn:
    result = await conn.execute(select(table))
```

Added in version 1.4.

Member Name	Description
<code>aclose()</code>	A synonym for <code>AsyncConnection.close()</code> .
<code>begin()</code>	Begin a transaction prior to <code>autobegin</code> occurring.
<code>begin_nested()</code>	Begin a nested transaction and return a transaction handle.
<code>close()</code>	Close this <code>AsyncConnection</code> .
<code>commit()</code>	Commit the transaction that is currently in progress.
<code>exec_driver_sql()</code>	Executes a driver-level SQL string and return buffered <code>Result</code> .
<code>execute()</code>	Executes a SQL statement construct and return a buffered <code>Result</code> .
<code>execution_options()</code>	Set non-SQL options for the connection which take effect during execution.
<code>get_nested_transaction()</code>	Return an <code>AsyncTransaction</code> representing the current nested (savepoint) transaction, if any.
<code>get_raw_connection()</code>	Return the pooled DBAPI-level connection in use by this <code>AsyncConnection</code> .
<code>get_transaction()</code>	Return an <code>AsyncTransaction</code> representing the current transaction, if any.
<code>in_nested_transaction()</code>	Return True if a transaction is in progress.
<code>in_transaction()</code>	Return True if a transaction is in progress.
<code>info</code>	Return the <code>Connection.info</code> dictionary of the underlying <code>Connection</code> .
<code>invalidate()</code>	Invalidate the underlying DBAPI connection associated with this <code>Connection</code> .
<code>rollback()</code>	Roll back the transaction that is currently in progress.
<code>run_sync()</code>	Invoke the given synchronous (i.e. not <code>async</code>) callable, passing a synchronous-style <code>Connection</code> as the first argument.
<code>scalar()</code>	Executes a SQL statement construct and returns a scalar object.
<code>scalars()</code>	Executes a SQL statement construct and returns a scalar objects.
<code>start()</code>	Start this <code>AsyncConnection</code> object's context outside of using a Python <code>with:</code> block.
<code>stream()</code>	Execute a statement and return an awaitable yielding a <code>AsyncResult</code> object.
<code>stream_scalars()</code>	Execute a statement and return an awaitable yielding a <code>AsyncScalarResult</code> object.
<code>sync_connection</code>	Reference to the sync-style <code>Connection</code> this <code>AsyncConnection</code> proxies requests towards.

<code>sync_engine</code>	Reference to the sync-style Engine this AsyncConnection is associated with via its underlying Connection .
--------------------------	---

method `sqlalchemy.ext.asyncio.AsyncConnection.async aclose()` → None

A synonym for `AsyncConnection.close()`.

The `AsyncConnection.aclose()` name is specifically to support the Python standard library `@contextlib.aclosing` context manager function.

Added in version 2.0.20.

method `sqlalchemy.ext.asyncio.AsyncConnection.begin()` → `AsyncTransaction`

Begin a transaction prior to autobegin occurring.

method `sqlalchemy.ext.asyncio.AsyncConnection.begin_nested()` → `AsyncTransaction`

Begin a nested transaction and return a transaction handle.

method `sqlalchemy.ext.asyncio.AsyncConnection.async close()` → None

Close this **AsyncConnection**.

This has the effect of also rolling back the transaction if one is in place.

property closed: Any

Return True if this connection is closed.

Proxied for the **Connection** class on behalf of the **AsyncConnection** class.

method `sqlalchemy.ext.asyncio.AsyncConnection.async commit()` → None

Commit the transaction that is currently in progress.

This method commits the current transaction if one has been started. If no transaction was started, the method has no effect, assuming the connection is in a non-invalidated state.

A transaction is begun on a **Connection** automatically whenever a statement is first executed, or when the `Connection.begin()` method is called.

property connection: NoReturn

Not implemented for async; call `AsyncConnection.get_raw_connection()`.

property default_isolation_level: Any

The initial-connection time isolation level associated with the **Dialect** in use.

Proxied for the **Connection** class on behalf of the **AsyncConnection** class.

This value is independent of the `Connection.execution_options.isolation_level` and `Engine.execution_options.isolation_level` execution options, and is determined by the **Dialect** when the first connection is created, by performing a SQL query against the database for the current isolation level before any additional commands have been emitted.

Calling this accessor does not invoke any new SQL queries.

See also

`Connection.get_isolation_level()` - view current actual isolation level

`create_engine.isolation_level` - set per **Engine** isolation level

`Connection.execution_options.isolation_level` - set per **Connection** isolation level

property dialect: Dialect

Proxy for the `Connection.dialect` attribute on behalf of the **AsyncConnection** class.

method `sqlalchemy.ext.asyncio.AsyncConnection.async exec_driver_sql(statement: str, parameters: _DBAPIAnyExecuteParams / None = None, execution_options: CoreExecuteOptionsParameter / None = None)` → `CursorResult[Any]`

Executes a driver-level SQL string and return buffered **Result**.

method `sqlalchemy.ext.asyncio.AsyncConnection.async execute(statement: Executable, parameters: _CoreAnyExecuteParams / None = None, *, execution_options: CoreExecuteOptionsParameter / None = None)` → `CursorResult[Unpack[TupleAny]]`

Executes a SQL statement construct and return a buffered **Result**.

Parameters:

- **object** –

The statement to be executed. This is always an object that is in both the **ClauseElement** and **Executable** hierarchies, including:

- **Select**
- **Insert, Update, Delete**
- **TextClause** and **TextualSelect**
- **DDL** and objects which inherit from **ExecutableDDLElement**
- **parameters** – parameters which will be bound into the statement. This may be either a dictionary of parameter names to values, or a mutable sequence (e.g. a list) of dictionaries. When a list of dictionaries is passed, the underlying statement execution will make use of the DBAPI `cursor.executemany()` method. When a single dictionary is passed, the DBAPI `cursor.execute()` method will be used.
- **execution_options** – optional dictionary of execution options, which will be associated with the statement execution. This dictionary can provide a subset of the options that are accepted by **Connection.execution_options()**.

Returns:

a **Result** object.

method `sqlalchemy.ext.asyncio.AsyncConnection.async execution_options(**opt: Any) → AsyncConnection`

Set non-SQL options for the connection which take effect during execution.

This returns this **AsyncConnection** object with the new options added.

See **Connection.execution_options()** for full details on this method.

method `sqlalchemy.ext.asyncio.AsyncConnection.get_nested_transaction() → AsyncTransaction | None`

Return an **AsyncTransaction** representing the current nested (savepoint) transaction, if any.

This makes use of the underlying synchronous connection's **Connection.get_nested_transaction()** method to get the current **Transaction**, which is then proxied in a new **AsyncTransaction** object.

Added in version 1.4.0b2.

method `sqlalchemy.ext.asyncio.AsyncConnection.async get_raw_connection() → PoolProxiedConnection`

Return the pooled DBAPI-level connection in use by this **AsyncConnection**.

This is a SQLAlchemy connection-pool proxied connection which then has the attribute `_ConnectionFairy.driver_connection` that refers to the actual driver connection. Its `_ConnectionFairy.dbapi_connection` refers instead to an **AdaptedConnection** instance that adapts the driver connection to the DBAPI protocol.

method `sqlalchemy.ext.asyncio.AsyncConnection.get_transaction() → AsyncTransaction | None`

Return an **AsyncTransaction** representing the current transaction, if any.

This makes use of the underlying synchronous connection's **Connection.get_transaction()** method to get the current **Transaction**, which is then proxied in a new **AsyncTransaction** object.

Added in version 1.4.0b2.

method `sqlalchemy.ext.asyncio.AsyncConnection.in_nested_transaction() → bool`

Return True if a transaction is in progress.

Added in version 1.4.0b2.

method `sqlalchemy.ext.asyncio.AsyncConnection.in_transaction() → bool`

Return True if a transaction is in progress.

attribute `sqlalchemy.ext.asyncio.AsyncConnection.info`

Return the **Connection.info** dictionary of the underlying **Connection**.

This dictionary is freely writable for user-defined state to be associated with the database connection.

This attribute is only available if the **AsyncConnection** is currently connected. If the **AsyncConnection.closed** attribute is `True`, then accessing this attribute will raise **ResourceClosedError**.

Added in version 1.4.0b2.

method `sqlalchemy.ext.asyncio.AsyncConnection`. async `invalidate`(*exception: BaseException / None = None*) → None

Invalidate the underlying DBAPI connection associated with this `Connection`.

See the method `Connection.invalidate()` for full detail on this method.

property `invalidated`: Any

Return True if this connection was invalidated.

Proxied for the `Connection` class on behalf of the `AsyncConnection` class.

This does not indicate whether or not the connection was invalidated at the pool level, however

method `sqlalchemy.ext.asyncio.AsyncConnection`. async `rollback`() → None

Roll back the transaction that is currently in progress.

This method rolls back the current transaction if one has been started. If no transaction was started, the method has no effect. If a transaction was started and the connection is in an invalidated state, the transaction is cleared using this method.

A transaction is begun on a `Connection` automatically whenever a statement is first executed, or when the `Connection.begin()` method is called.

method `sqlalchemy.ext.asyncio.AsyncConnection`. async `run_sync`(*fn: ~typing.Callable[~typing.Concatenate[~sqlalchemy.engine.base.Connection, ~_P]], ~sqlalchemy.ext.asyncio.engine._T], *arg: ~typing._P, **kw: ~typing._P*) → _T

Invoke the given synchronous (i.e. not async) callable, passing a synchronous-style `Connection` as the first argument.

This method allows traditional synchronous SQLAlchemy functions to run within the context of an asyncio application.

E.g.:

```
def do_something_with_core(conn: Connection, arg1: int, arg2: str) -> str:
    """A synchronous function that does not require awaiting

    :param conn: a Core SQLAlchemy Connection, used synchronously

    :return: an optional return value is supported

    """
    conn.execute(some_table.insert().values(int_col=arg1, str_col=arg2))
    return "success"

async def do_something_async(async_engine: AsyncEngine) -> None:
    """an async function that uses awaiting"""

    async with async_engine.begin() as async_conn:
        # run do_something_with_core() with a sync-style
        # Connection, proxied into an awaitable
        return_code = await async_conn.run_sync(
            do_something_with_core, 5, "strval"
        )
        print(return_code)
```

This method maintains the asyncio event loop all the way through to the database connection by running the given callable in a specially instrumented greenlet.

The most rudimentary use of `AsyncConnection.run_sync()` is to invoke methods such as `MetaData.create_all()`, given an `AsyncConnection` that needs to be provided to `MetaData.create_all()` as a `Connection` object:

```
# run metadata.create_all(conn) with a sync-style Connection,
# proxied into an awaitable
with async_engine.begin() as conn:
    await conn.run_sync(metadata.create_all)
```

Note

The provided callable is invoked inline within the asyncio event loop, and will block on traditional IO calls. IO within this callable should only call into SQLAlchemy’s asyncio database APIs which will be properly adapted to the greenlet context.

See also

`AsyncSession.run_sync()`

Running Synchronous Methods and Functions under asyncio

method `sqlalchemy.ext.asyncio.AsyncConnection`. async `scalar`(statement: *Executable*, parameters: *_CoreSingleExecuteParams* | *None* = *None*, *, execution_options: *CoreExecuteOptionsParameter* | *None* = *None*) → Any

Executes a SQL statement construct and returns a scalar object.

This method is shorthand for invoking the `Result.scalar()` method after invoking the `Connection.execute()` method. Parameters are equivalent.

Returns:

a scalar Python value representing the first column of the first row returned.

method `sqlalchemy.ext.asyncio.AsyncConnection`. async `scalars`(statement: *Executable*, parameters: *_CoreAnyExecuteParams* | *None* = *None*, *, execution_options: *CoreExecuteOptionsParameter* | *None* = *None*) → `ScalarResult`[Any]

Executes a SQL statement construct and returns a scalar objects.

This method is shorthand for invoking the `Result.scalars()` method after invoking the `Connection.execute()` method. Parameters are equivalent.

Returns:

a `ScalarResult` object.

Added in version 1.4.24.

method `sqlalchemy.ext.asyncio.AsyncConnection`. async `start`(is_ctxmanager: bool = False) → `AsyncConnection`

Start this `AsyncConnection` object’s context outside of using a Python `with`: block.

method `sqlalchemy.ext.asyncio.AsyncConnection`. `stream`(statement: *Executable*, parameters: *_CoreAnyExecuteParams* | *None* = *None*, *, execution_options: *CoreExecuteOptionsParameter* | *None* = *None*) → `AsyncIterator`[`AsyncResult`[`Unpack`[`TupleAny`]]]

Execute a statement and return an awaitable yielding a `AsyncResult` object.

E.g.:

```
result = await conn.stream(stmt)
async for row in result:
    print(f"{row}")
```

The `AsyncConnection.stream()` method supports optional context manager use against the `AsyncResult` object, as in:

```
async with conn.stream(stmt) as result:
    async for row in result:
        print(f"{row}")
```

In the above pattern, the `AsyncResult.close()` method is invoked unconditionally, even if the iterator is interrupted by an exception throw. Context manager use remains optional, however, and the function may be called in either an `async with fn():` or `await fn()` style.

Added in version 2.0.0b3: added context manager support

Returns:

an awaitable object that will yield an `AsyncResult` object.

See also

`AsyncConnection.stream_scalars()`

method `sqlalchemy.ext.asyncio.AsyncConnection`. `stream_scalars`(statement: *Executable*, parameters: *_CoreSingleExecuteParams* | *None* = *None*, *, execution_options: *CoreExecuteOptionsParameter* | *None* = *None*) → `AsyncIterator`[`AsyncScalarResult`[Any]]

Execute a statement and return an awaitable yielding a `AsyncScalarResult` object.

E.g.:

```
result = await conn.stream_scalars(stmt)
async for scalar in result:
    print(f"{scalar}")
```

This method is shorthand for invoking the `AsyncResult.scalars()` method after invoking the `Connection.stream()` method. Parameters are equivalent.

The `AsyncConnection.stream_scalars()` method supports optional context manager use against the `AsyncScalarResult` object, as in:

```
async with conn.stream_scalars(stmt) as result:
    async for scalar in result:
        print(f"{scalar}")
```

In the above pattern, the `AsyncScalarResult.close()` method is invoked unconditionally, even if the iterator is interrupted by an exception throw. Context manager use remains optional, however, and the function may be called in either an `async with fn():` or `await fn()` style.

Added in version 2.0.0b3: added context manager support

Returns:

an awaitable object that will yield an `AsyncScalarResult` object.

Added in version 1.4.24.

See also

`AsyncConnection.stream()`

attribute `sqlalchemy.ext.asyncio.AsyncConnection.sync_connection`: `Connection` | None

Reference to the sync-style `Connection` this `AsyncConnection` proxies requests towards.

This instance can be used as an event target.

See also

Using events with the asyncio extension

attribute `sqlalchemy.ext.asyncio.AsyncConnection.sync_engine`: `Engine`

Reference to the sync-style `Engine` this `AsyncConnection` is associated with via its underlying `Connection`.

This instance can be used as an event target.

See also

Using events with the asyncio extension

class `sqlalchemy.ext.asyncio.AsyncTransaction`

inherits from `sqlalchemy.ext.asyncio.base.ProxyComparable`, `sqlalchemy.ext.asyncio.base.StartableContext`

An asyncio proxy for a `Transaction`.

Member Name	Description
<code>close()</code>	Close this <code>AsyncTransaction</code> .
<code>commit()</code>	Commit this <code>AsyncTransaction</code> .
<code>rollback()</code>	Roll back this <code>AsyncTransaction</code> .
<code>start()</code>	Start this <code>AsyncTransaction</code> object's context outside of using a Python <code>with:</code> block.

method `sqlalchemy.ext.asyncio.AsyncTransaction.async close()` → None

Close this `AsyncTransaction`.

If this transaction is the base transaction in a begin/commit nesting, the transaction will `rollback()`. Otherwise, the method returns.

This is used to cancel a Transaction without affecting the scope of an enclosing transaction.

method `sqlalchemy.ext.asyncio.AsyncTransaction.async commit()` → None

Commit this `AsyncTransaction`.

method `sqlalchemy.ext.asyncio.AsyncTransaction.async rollback()` → None

Roll back this `AsyncTransaction`.

method `sqlalchemy.ext.asyncio.AsyncTransaction.async start(is_ctxmanager: bool = False)` → `AsyncTransaction`

Start this `AsyncTransaction` object’s context outside of using a Python `with:` block.

Result Set API Documentation

The `AsyncResult` object is an async-adapted version of the `Result` object. It is only returned when using the `AsyncConnection.stream()` or `AsyncSession.stream()` methods, which return a result object that is on top of an active database cursor.

Object Name	Description
<code>AsyncMappingResult</code>	A wrapper for a <code>AsyncResult</code> that returns dictionary values rather than <code>Row</code> values.
<code>AsyncResult</code>	An asyncio wrapper around a <code>Result</code> object.
<code>AsyncScalarResult</code>	A wrapper for a <code>AsyncResult</code> that returns scalar values rather than <code>Row</code> values.
<code>AsyncTupleResult</code>	A <code>AsyncResult</code> that’s typed as returning plain Python tuples instead of rows.

class `sqlalchemy.ext.asyncio.AsyncResult`

inherits from `sqlalchemy.engine._WithKeys`, `sqlalchemy.ext.asyncio.AsyncCommon`

An asyncio wrapper around a `Result` object.

The `AsyncResult` only applies to statement executions that use a server-side cursor. It is returned only from the `AsyncConnection.stream()` and `AsyncSession.stream()` methods.

Note

As is the case with `Result`, this object is used for ORM results returned by `AsyncSession.execute()`, which can yield instances of ORM mapped objects either individually or within tuple-like rows. Note that these result objects do not deduplicate instances or rows automatically as is the case with the legacy `Query` object. For in-Python de-duplication of instances or rows, use the `AsyncResult.unique()` modifier method.

Added in version 1.4.

Member Name	Description
<code>all()</code>	Return all rows in a list.
<code>close()</code>	Close this result.
<code>columns()</code>	Establish the columns that should be returned in each row.
<code>fetchall()</code>	A synonym for the <code>AsyncResult.all()</code> method.
<code>fetchmany()</code>	Fetch many rows.
<code>fetchone()</code>	Fetch one row.
<code>first()</code>	Fetch the first row or <code>None</code> if no row is present.
<code>freeze()</code>	Return a callable object that will produce copies of this <code>AsyncResult</code> when invoked.
<code>keys()</code>	Return an iterable view which yields the string keys that would be represented by each <code>Row</code> .
<code>mappings()</code>	Apply a mappings filter to returned rows, returning an instance of <code>AsyncMappingResult</code> .
<code>one()</code>	Return exactly one row or raise an exception.
<code>one_or_none()</code>	Return at most one result or raise an exception.
<code>partitions()</code>	Iterate through sub-lists of rows of the size given.
<code>scalar()</code>	Fetch the first column of the first row, and close the result set.
<code>scalar_one()</code>	Return exactly one scalar result or raise an exception.
<code>scalar_one_or_none()</code>	Return exactly one scalar result or <code>None</code> .
<code>scalars()</code>	Return an <code>AsyncScalarResult</code> filtering object which will return single elements rather than <code>Row</code> objects.

<code>tuples()</code>	Apply a “typed tuple” typing filter to returned rows.
<code>unique()</code>	Apply unique filtering to the objects returned by this AsyncResult .
<code>yield_per()</code>	Configure the row-fetching strategy to fetch <code>num</code> rows at a time.

method `sqlalchemy.ext.asyncio.AsyncResult`.async **all**() → Sequence[Row[Unpack[_Ts]]]

- Return all rows in a list.
- Closes the result set after invocation. Subsequent invocations will return an empty list.

Returns:

a list of **Row** objects.

method `sqlalchemy.ext.asyncio.AsyncResult`.async **close**() → None

inherited from the `AsyncCommon.close()` method of `AsyncCommon`

Close this result.

property **closed**: bool

proxies the `.closed` attribute of the underlying result object, if any, else raises `AttributeError`.

Added in version 2.0.0b3.

method `sqlalchemy.ext.asyncio.AsyncResult`.**columns**(*col_expressions: _KeyIndexType) → Self

- Establish the columns that should be returned in each row.
- Refer to **Result.columns()** in the synchronous SQLAlchemy API for a complete behavioral description.

method `sqlalchemy.ext.asyncio.AsyncResult`.async **fetchall**() → Sequence[Row[Unpack[_Ts]]]

A synonym for the **AsyncResult.all()** method.

Added in version 2.0.

method `sqlalchemy.ext.asyncio.AsyncResult`.async **fetchmany**(size: int / None = None) → Sequence[Row[Unpack[_Ts]]]

- Fetch many rows.
- When all rows are exhausted, returns an empty list.
- This method is provided for backwards compatibility with SQLAlchemy 1.x.x.
- To fetch rows in groups, use the **AsyncResult.partitions()** method.

Returns:

a list of **Row** objects.

See also

AsyncResult.partitions()

method `sqlalchemy.ext.asyncio.AsyncResult`.async **fetchone**() → Row[Unpack[_Ts]] | None

- Fetch one row.
- When all rows are exhausted, returns None.
- This method is provided for backwards compatibility with SQLAlchemy 1.x.x.
- To fetch the first row of a result only, use the **AsyncResult.first()** method. To iterate through all rows, iterate the **AsyncResult** object directly.

Returns:

a **Row** object if no filters are applied, or `None` if no rows remain.

method `sqlalchemy.ext.asyncio.AsyncResult`.async **first**() → Row[Unpack[_Ts]] | None

Fetch the first row or `None` if no row is present.

Closes the result set and discards remaining rows.

Note

This method returns one **row**, e.g. tuple, by default. To return exactly one single scalar value, that is, the first column of the first row, use the **`AsyncResult.scalar()`** method, or combine **`AsyncResult.scalars()`** and **`AsyncResult.first()`**.

Additionally, in contrast to the behavior of the legacy ORM **`Query.first()`** method, **no limit is applied** to the SQL query which was invoked to produce this **`AsyncResult`**; for a DBAPI driver that buffers results in memory before yielding rows, all rows will be sent to the Python process and all but the first row will be discarded.

See also

[ORM Query Unified with Core Select](#)

Returns:

a **Row** object, or `None` if no rows remain.

See also

`AsyncResult.scalar()`

`AsyncResult.one()`

method **`sqlalchemy.ext.asyncio.AsyncResult`**.`async freeze()` → **`FrozenResult`**[`Unpack[_Ts]`]

Return a callable object that will produce copies of this **`AsyncResult`** when invoked.

The callable object returned is an instance of **`FrozenResult`**.

This is used for result set caching. The method must be called on the result when it has been unconsumed, and calling the method will consume the result fully. When the **`FrozenResult`** is retrieved from a cache, it can be called any number of times where it will produce a new **`Result`** object each time against its stored set of rows.

See also

[Re-Executing Statements](#) - example usage within the ORM to implement a result-set cache.

method **`sqlalchemy.ext.asyncio.AsyncResult`**.`keys()` → **`RMKeyView`**

inherited from the `sqlalchemy.engine._WithKeys.keys` method of `sqlalchemy.engine._WithKeys`

Return an iterable view which yields the string keys that would be represented by each **Row**.

The keys can represent the labels of the columns returned by a core statement or the names of the orm classes returned by an orm execution.

The view also can be tested for key containment using the Python `in` operator, which will test both for the string keys represented in the view, as well as for alternate keys such as column objects.

Changed in version 1.4: a key view object is returned rather than a plain list.

method **`sqlalchemy.ext.asyncio.AsyncResult`**.`mappings()` → **`AsyncMappingResult`**

Apply a mappings filter to returned rows, returning an instance of **`AsyncMappingResult`**.

When this filter is applied, fetching rows will return **`RowMapping`** objects instead of **Row** objects.

Returns:

a new **`AsyncMappingResult`** filtering object referring to the underlying **`Result`** object.

method **`sqlalchemy.ext.asyncio.AsyncResult`**.`async one()` → **`Row`**[`Unpack[_Ts]`]

Return exactly one row or raise an exception.

Raises **`NoResultFound`** if the result returns no rows, or **`MultipleResultsFound`** if multiple rows would be returned.

Note

This method returns one **row**, e.g. tuple, by default. To return exactly one single scalar value, that is, the first column of the first row, use the **`AsyncResult.scalar_one()`** method, or combine **`AsyncResult.scalars()`** and **`AsyncResult.one()`**.

Added in version 1.4.

Returns:

The first `Row`.

Raises:

`MultipleResultsFound`, `NoResultFound`

See also

`AsyncResult.first()`

`AsyncResult.one_or_none()`

`AsyncResult.scalar_one()`

method `sqlalchemy.ext.asyncio.AsyncResult`. async `one_or_none()` → `Row[Unpack[_Ts]]` | `None`

Return at most one result or raise an exception.

Returns `None` if the result has no rows. Raises `MultipleResultsFound` if multiple rows are returned.

Added in version 1.4.

Returns:

The first `Row` or `None` if no row is available.

Raises:

`MultipleResultsFound`

See also

`AsyncResult.first()`

`AsyncResult.one()`

method `sqlalchemy.ext.asyncio.AsyncResult`. async `partitions`(*size: int* / *None = None*) → `AsyncIterator[Sequence[Row[Unpack[_Ts]]]]`

Iterate through sub-lists of rows of the size given.

An async iterator is returned:

```
async def scroll_results(connection):
    result = await connection.stream(select(users_table))

    async for partition in result.partitions(100):
        print("list of rows: %s" % partition)
```

Refer to `Result.partitions()` in the synchronous SQLAlchemy API for a complete behavioral description.

method `sqlalchemy.ext.asyncio.AsyncResult`. async `scalar()` → `Any`

Fetch the first column of the first row, and close the result set.

Returns `None` if there are no rows to fetch.

No validation is performed to test if additional rows remain.

After calling this method, the object is fully closed, e.g. the `CursorResult.close()` method will have been called.

Returns:

a Python scalar value, or `None` if no rows remain.

method `sqlalchemy.ext.asyncio.AsyncResult.async scalar_one()` → Any

Return exactly one scalar result or raise an exception.

This is equivalent to calling `AsyncResult.scalars()` and then `AsyncScalarResult.one()`.

See also

`AsyncScalarResult.one()`

`AsyncResult.scalars()`

method `sqlalchemy.ext.asyncio.AsyncResult.async scalar_one_or_none()` → Any | None

Return exactly one scalar result or `None`.

This is equivalent to calling `AsyncResult.scalars()` and then `AsyncScalarResult.one_or_none()`.

See also

`AsyncScalarResult.one_or_none()`

`AsyncResult.scalars()`

method `sqlalchemy.ext.asyncio.AsyncResult.scalars(index: _KeyIndexType = 0)` → `AsyncScalarResult`[Any]

Return an `AsyncScalarResult` filtering object which will return single elements rather than `Row` objects.

Refer to `Result.scalars()` in the synchronous SQLAlchemy API for a complete behavioral description.

Parameters:

index – integer or row key indicating the column to be fetched from each row, defaults to `0` indicating the first column.

Returns:

a new `AsyncScalarResult` filtering object referring to this `AsyncResult` object.

property `t`: `AsyncTupleResult[Tuple[Unpack[_Ts]]]`

Apply a “typed tuple” typing filter to returned rows.

Deprecated since version 2.1.0: The `AsyncResult.t` attribute is deprecated, `Row` now behaves like a tuple and can unpack types directly.

The `AsyncResult.t` attribute is a synonym for calling the `AsyncResult.tuples()` method.

Added in version 2.0.

See also

`Row` now represents individual column types directly without `Tuple` - describes a migration path from this workaround for SQLAlchemy 2.1.

method `sqlalchemy.ext.asyncio.AsyncResult.tuples()` → `AsyncTupleResult`[`Tuple`[`Unpack[_Ts]`]]

Apply a “typed tuple” typing filter to returned rows.

Deprecated since version 2.1.0: The `AsyncResult.tuples()` method is deprecated, `Row` now behaves like a tuple and can unpack types directly.

This method returns the same `AsyncResult` object at runtime, however annotates as returning a `AsyncTupleResult` object that will indicate to **PEP 484** typing tools that plain typed `Tuple` instances are returned rather than rows. This allows tuple unpacking and `__getitem__` access of `Row` objects to be typed, for those cases where the statement invoked itself included typing information.

Added in version 2.0.

Returns:

the `AsyncTupleResult` type at typing time.

See also

Row now represents individual column types directly without Tuple - describes a migration path from this workaround for SQLAlchemy 2.1.

AsyncResult.t - shorter synonym

Row.t - Row version

method sqlalchemy.ext.asyncio.AsyncResult.unique(strategy: _UniqueFilterType | None = None) → Self

Apply unique filtering to the objects returned by this AsyncResult.

Refer to Result.unique() in the synchronous SQLAlchemy API for a complete behavioral description.

method sqlalchemy.ext.asyncio.AsyncResult.yield_per(num: int) → Self

inherited from the FilterResult.yield_per() method of FilterResult

Configure the row-fetching strategy to fetch num rows at a time.

The FilterResult.yield_per() method is a pass through to the Result.yield_per() method. See that method’s documentation for usage notes.

Added in version 1.4.40: - added FilterResult.yield_per() so that the method is available on all result set implementations

See also

Using Server Side Cursors (a.k.a. stream results) - describes Core behavior for Result.yield_per()

Fetching Large Result Sets with Yield Per - in the ORM Querying Guide

class sqlalchemy.ext.asyncio.AsyncScalarResult

inherits from sqlalchemy.ext.asyncio.AsyncCommon

A wrapper for a AsyncResult that returns scalar values rather than Row values.

The AsyncScalarResult object is acquired by calling the AsyncResult.scalars() method.

Refer to the ScalarResult object in the synchronous SQLAlchemy API for a complete behavioral description.

Added in version 1.4.

Member Name	Description
all()	Return all scalar values in a list.
close()	Close this result.
fetchall()	A synonym for the AsyncScalarResult.all() method.
fetchmany()	Fetch many objects.
first()	Fetch the first object or None if no object is present.
one()	Return exactly one object or raise an exception.
one_or_none()	Return at most one object or raise an exception.
partitions()	Iterate through sub-lists of elements of the size given.
unique()	Apply unique filtering to the objects returned by this AsyncScalarResult.
yield_per()	Configure the row-fetching strategy to fetch num rows at a time.

method sqlalchemy.ext.asyncio.AsyncScalarResult.async all() → Sequence[_R]

Return all scalar values in a list.

Equivalent to AsyncResult.all() except that scalar values, rather than Row objects, are returned.

method sqlalchemy.ext.asyncio.AsyncScalarResult.async close() → None

inherited from the AsyncCommon.close() method of AsyncCommon

Close this result.

property closed: bool

proxies the `.closed` attribute of the underlying result object, if any, else raises `AttributeError`.

Added in version 2.0.0b3.

method `sqlalchemy.ext.asyncio.AsyncScalarResult`.async **fetchall**() → Sequence[_R]

A synonym for the `AsyncScalarResult.all()` method.

method `sqlalchemy.ext.asyncio.AsyncScalarResult`.async **fetchmany**(*size: int | None = None*) → Sequence[_R]

Fetch many objects.

Equivalent to `AsyncResult.fetchmany()` except that scalar values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncScalarResult`.async **first**() → _R | None

Fetch the first object or `None` if no object is present.

Equivalent to `AsyncResult.first()` except that scalar values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncScalarResult`.async **one**() → _R

Return exactly one object or raise an exception.

Equivalent to `AsyncResult.one()` except that scalar values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncScalarResult`.async **one_or_none**() → _R | None

Return at most one object or raise an exception.

Equivalent to `AsyncResult.one_or_none()` except that scalar values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncScalarResult`.async **partitions**(*size: int | None = None*) → AsyncIterator[Sequence[_R]]

Iterate through sub-lists of elements of the size given.

Equivalent to `AsyncResult.partitions()` except that scalar values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncScalarResult`. **unique**(*strategy: _UniqueFilterType | None = None*) → Self

Apply unique filtering to the objects returned by this `AsyncScalarResult`.

See `AsyncResult.unique()` for usage details.

method `sqlalchemy.ext.asyncio.AsyncScalarResult`. **yield_per**(*num: int*) → Self

inherited from the `FilterResult.yield_per()` method of `FilterResult`

Configure the row-fetching strategy to fetch `num` rows at a time.

The `FilterResult.yield_per()` method is a pass through to the `Result.yield_per()` method. See that method’s documentation for usage notes.

Added in version 1.4.40: - added `FilterResult.yield_per()` so that the method is available on all result set implementations

See also

Using Server Side Cursors (a.k.a. stream results) - describes Core behavior for `Result.yield_per()`

Fetching Large Result Sets with Yield Per - in the ORM Querying Guide

class `sqlalchemy.ext.asyncio`. **AsyncMappingResult**

inherits from `sqlalchemy.engine._WithKeys`, `sqlalchemy.ext.asyncio.AsyncCommon`

A wrapper for a `AsyncResult` that returns dictionary values rather than `Row` values.

The `AsyncMappingResult` object is acquired by calling the `AsyncResult.mappings()` method.

Refer to the `MappingResult` object in the synchronous SQLAlchemy API for a complete behavioral description.

Added in version 1.4.

Member Name	Description
<code>all()</code>	Return all rows in a list.

<code>close()</code>	Close this result.
<code>columns()</code>	Establish the columns that should be returned in each row.
<code>fetchall()</code>	A synonym for the <code>AsyncMappingResult.all()</code> method.
<code>fetchmany()</code>	Fetch many rows.
<code>fetchone()</code>	Fetch one object.
<code>first()</code>	Fetch the first object or <code>None</code> if no object is present.
<code>keys()</code>	Return an iterable view which yields the string keys that would be represented by each <code>Row</code> .
<code>one()</code>	Return exactly one object or raise an exception.
<code>one_or_none()</code>	Return at most one object or raise an exception.
<code>partitions()</code>	Iterate through sub-lists of elements of the size given.
<code>unique()</code>	Apply unique filtering to the objects returned by this <code>AsyncMappingResult</code> .
<code>yield_per()</code>	Configure the row-fetching strategy to fetch <code>num</code> rows at a time.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.async all()` → Sequence[`RowMapping`]

Return all rows in a list.

Equivalent to `AsyncResult.all()` except that `RowMapping` values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.async close()` → None

inherited from the `AsyncCommon.close()` method of `AsyncCommon`

Close this result.

property `closed`: bool

proxies the `.closed` attribute of the underlying result object, if any, else raises `AttributeError`.

Added in version 2.0.0b3.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.columns(*col_expressions: _KeyIndexType)` → Self

Establish the columns that should be returned in each row.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.async fetchall()` → Sequence[`RowMapping`]

A synonym for the `AsyncMappingResult.all()` method.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.async fetchmany(size: int | None = None)` → Sequence[`RowMapping`]

Fetch many rows.

Equivalent to `AsyncResult.fetchmany()` except that `RowMapping` values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.async fetchone()` → `RowMapping` | None

Fetch one object.

Equivalent to `AsyncResult.fetchone()` except that `RowMapping` values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.async first()` → `RowMapping` | None

Fetch the first object or `None` if no object is present.

Equivalent to `AsyncResult.first()` except that `RowMapping` values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.keys()` → `RMKeyView`

inherited from the `sqlalchemy.engine._WithKeys.keys` method of `sqlalchemy.engine._WithKeys`

Return an iterable view which yields the string keys that would be represented by each `Row`.

The keys can represent the labels of the columns returned by a core statement or the names of the orm classes returned by an orm execution.

The view also can be tested for key containment using the Python `in` operator, which will test both for the string keys represented in the view, as well as for alternate keys such as column objects.

Changed in version 1.4: a key view object is returned rather than a plain list.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.async one()` → `RowMapping`

Return exactly one object or raise an exception.

Equivalent to `AsyncResult.one()` except that `RowMapping` values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.async one_or_none()` → `RowMapping` | `None`

Return at most one object or raise an exception.

Equivalent to `AsyncResult.one_or_none()` except that `RowMapping` values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.async partitions(size: int / None = None)` → `AsyncIterator[Sequence[RowMapping]]`

Iterate through sub-lists of elements of the size given.

Equivalent to `AsyncResult.partitions()` except that `RowMapping` values, rather than `Row` objects, are returned.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.unique(strategy: _UniqueFilterType / None = None)` → `Self`

Apply unique filtering to the objects returned by this `AsyncMappingResult`.

See `AsyncResult.unique()` for usage details.

method `sqlalchemy.ext.asyncio.AsyncMappingResult.yield_per(num: int)` → `Self`

inherited from the `FilterResult.yield_per()` method of `FilterResult`

Configure the row-fetching strategy to fetch `num` rows at a time.

The `FilterResult.yield_per()` method is a pass through to the `Result.yield_per()` method. See that method’s documentation for usage notes.

Added in version 1.4.40: - added `FilterResult.yield_per()` so that the method is available on all result set implementations

See also

Using Server Side Cursors (a.k.a. stream results) - describes Core behavior for `Result.yield_per()`

Fetching Large Result Sets with Yield Per - in the ORM Querying Guide

class `sqlalchemy.ext.asyncio.AsyncTupleResult`

inherits from `sqlalchemy.ext.asyncio.AsyncCommon`, `sqlalchemy.util.langhelpers.TypingOnly`

A `AsyncResult` that’s typed as returning plain Python tuples instead of rows.

Since `Row` acts like a tuple in every way already, this class is a typing only class, regular `AsyncResult` is still used at runtime.

ORM Session API Documentation

Object Name	Description
<code>async_object_session</code> (instance)	Return the <code>AsyncSession</code> to which the given instance belongs.
<code>async_scoped_session</code>	Provides scoped management of <code>AsyncSession</code> objects.
<code>async_session</code> (session)	Return the <code>AsyncSession</code> which is proxying the given <code>Session</code> object, if any.
<code>async_sessionmaker</code>	A configurable <code>AsyncSession</code> factory.
<code>AsyncAttrs</code>	Mixin class which provides an awaitable accessor for all attributes.
<code>AsyncSession</code>	Asyncio version of <code>Session</code> .
<code>AsyncSessionTransaction</code>	A wrapper for the ORM <code>SessionTransaction</code> object.
<code>close_all_sessions</code> ()	Close all <code>AsyncSession</code> sessions.

function `sqlalchemy.ext.asyncio.async_object_session(instance: object)` → `AsyncSession` | `None`

Return the `AsyncSession` to which the given instance belongs.

This function makes use of the sync-API function `object_session` to retrieve the `Session` which refers to the given instance, and from there links it to the original `AsyncSession`.

If the **AsyncSession** has been garbage collected, the return value is `None`.

This functionality is also available from the **InstanceState.async_session** accessor.

Parameters:

instance – an ORM mapped instance

Returns:

an **AsyncSession** object, or `None`.

Added in version 1.4.18.

function sqlalchemy.ext.asyncio.**async_session**(session: *Session*) → AsyncSession | None

Return the **AsyncSession** which is proxying the given **Session** object, if any.

Parameters:

session – a **Session** instance.

Returns:

a **AsyncSession** instance, or `None`.

Added in version 1.4.18.

function async sqlalchemy.ext.asyncio.**close_all_sessions**() → None

Close all **AsyncSession** sessions.

Added in version 2.0.23.

See also

close_all_sessions()

class sqlalchemy.ext.asyncio.**async_sessionmaker**

inherits from typing.Generic

A configurable **AsyncSession** factory.

The **async_sessionmaker** factory works in the same way as the **sessionmaker** factory, to generate new **AsyncSession** objects when called, creating them given the configurational arguments established here.

e.g.:

```
from sqlalchemy.ext.asyncio import create_async_engine
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.ext.asyncio import async_sessionmaker

async def run_some_sql(
    async_session: async_sessionmaker[AsyncSession],
) -> None:
    async with async_session() as session:
        session.add(SomeObject(data="object"))
        session.add(SomeOtherObject(name="other object"))
        await session.commit()

async def main() -> None:
    # an AsyncEngine, which the AsyncSession will use for connection
    # resources
    engine = create_async_engine(
        "postgresql+asyncpg://scott:tiger@localhost/"
    )

    # create a reusable factory for new AsyncSession instances
    async_session = async_sessionmaker(engine)

    await run_some_sql(async_session)
```



```
await engine.dispose()
```

The `async_sessionmaker` is useful so that different parts of a program can create new `AsyncSession` objects with a fixed configuration established up front. Note that `AsyncSession` objects may also be instantiated directly when not using `async_sessionmaker`.

Added in version 2.0: `async_sessionmaker` provides a `sessionmaker` class that’s dedicated to the `AsyncSession` object, including pep-484 typing support.

See also

Synopsis - ORM - shows example use

`sessionmaker` - general overview of the `sessionmaker` architecture

Opening and Closing a Session - introductory text on creating sessions using `sessionmaker`.

Member Name	Description
<code>__call__()</code>	Produce a new <code>AsyncSession</code> object using the configuration established in this <code>async_sessionmaker</code> .
<code>__init__()</code>	Construct a new <code>async_sessionmaker</code> .
<code>begin()</code>	Produce a context manager that both provides a new <code>AsyncSession</code> as well as a transaction that commits.
<code>configure()</code>	(Re)configure the arguments for this <code>async_sessionmaker</code> .

method `sqlalchemy.ext.asyncio.async_sessionmaker.__call__(**local_kw: Any) → _AS`

Produce a new `AsyncSession` object using the configuration established in this `async_sessionmaker`.

In Python, the `__call__` method is invoked on an object when it is “called” in the same way as a function:

```
AsyncSession = async_sessionmaker(async_engine, expire_on_commit=False)
session = AsyncSession() # invokes sessionmaker.__call__()
```

method `sqlalchemy.ext.asyncio.async_sessionmaker.__init__(bind: _AsyncSessionBind | None = None, *, class_: Type[_AS] = <class 'sqlalchemy.ext.asyncio.session.AsyncSession'>, autoflush: bool = True, expire_on_commit: bool = True, info: _InfoType | None = None, **kw: Any)`

Construct a new `async_sessionmaker`.

All arguments here except for `class_` correspond to arguments accepted by `Session` directly. See the `AsyncSession.__init__()` docstring for more details on parameters.

method `sqlalchemy.ext.asyncio.async_sessionmaker.begin() → _AsyncSessionContextManager[_AS]`

Produce a context manager that both provides a new `AsyncSession` as well as a transaction that commits.

e.g.:

```
async def main():
    Session = async_sessionmaker(some_engine)

    async with Session.begin() as session:
        session.add(some_object)

    # commits transaction, closes session
```

method `sqlalchemy.ext.asyncio.async_sessionmaker.configure(**new_kw: Any) → None`

(Re)configure the arguments for this `async_sessionmaker`.

e.g.:

```
AsyncSession = async_sessionmaker(some_engine)

AsyncSession.configure(bind=create_async_engine("sqlite+aiosqlite://"))
```

class `sqlalchemy.ext.asyncio.async_scoped_session`

inherits from `typing.Generic`

Provides scoped management of **AsyncSession** objects.

See the section **Using asyncio scoped session** for usage details.

Added in version 1.4.19.

Member Name	Description
<code>__call__()</code>	Return the current AsyncSession , creating it using the <code>scoped_session.session_factory</code> if not present.
<code>__init__()</code>	Construct a new async_scoped_session .
<code>aclose()</code>	A synonym for AsyncSession.close() .
<code>add()</code>	Place an object into this Session .
<code>add_all()</code>	Add the given collection of instances to this Session .
<code>begin()</code>	Return an AsyncSessionTransaction object.
<code>begin_nested()</code>	Return an AsyncSessionTransaction object which will begin a “nested” transaction, e.g. SAVEPOINT.
<code>close()</code>	Close out the transactional resources and ORM objects used by this AsyncSession .
<code>close_all()</code>	Close all AsyncSession sessions.
<code>commit()</code>	Commit the current transaction in progress.
<code>configure()</code>	reconfigure the sessionmaker used by this scoped_session .
<code>connection()</code>	Return a AsyncConnection object corresponding to this Session object’s transactional state.
<code>delete()</code>	Mark an instance as deleted.
<code>delete_all()</code>	Calls AsyncSession.delete() on multiple instances.
<code>execute()</code>	Execute a statement and return a buffered Result object.
<code>expire()</code>	Expire the attributes on an instance.
<code>expire_all()</code>	Expires all persistent instances within this Session.
<code>expunge()</code>	Remove the <i>instance</i> from this Session .
<code>expunge_all()</code>	Remove all object instances from this Session .
<code>flush()</code>	Flush all the object changes to the database.
<code>get()</code>	Return an instance based on the given primary key identifier, or None if not found.
<code>get_bind()</code>	Return a “bind” to which the synchronous proxied Session is bound.
<code>get_one()</code>	Return an instance based on the given primary key identifier, or raise an exception if not found.
<code>identity_key()</code>	Return an identity key.
<code>invalidate()</code>	Close this Session, using connection invalidation.
<code>is_modified()</code>	Return True if the given instance has locally modified attributes.
<code>merge()</code>	Copy the state of a given instance into a corresponding instance within this AsyncSession .
<code>merge_all()</code>	Calls AsyncSession.merge() on multiple instances.
<code>object_session()</code>	Return the Session to which an object belongs.
<code>refresh()</code>	Expire and refresh the attributes on the given instance.
<code>remove()</code>	Dispose of the current AsyncSession , if present.
<code>reset()</code>	Close out the transactional resources and ORM objects used by this Session , resetting the session to its initial state.
<code>rollback()</code>	Rollback the current transaction in progress.
<code>scalar()</code>	Execute a statement and return a scalar result.

scalars()	Execute a statement and return scalar results.
session_factory	The <i>session_factory</i> provided to <i>__init__</i> is stored in this attribute and may be accessed at a later time. This can be useful when a new non-scoped AsyncSession is needed.
stream()	Execute a statement and return a streaming AsyncResult object.
stream_scalars()	Execute a statement and return a stream of scalar results.

method **sqlalchemy.ext.asyncio.async_scoped_session.__call__**(***kw: Any*) → *_AS*

Return the current **AsyncSession**, creating it using the **scoped_session.session_factory** if not present.

Parameters:

****kw** – Keyword arguments will be passed to the **scoped_session.session_factory** callable, if an existing **AsyncSession** is not present. If the **AsyncSession** is present and keyword arguments have been passed, **InvalidRequestError** is raised.

method **sqlalchemy.ext.asyncio.async_scoped_session.__init__**(*session_factory: async_sessionmaker[_AS], scopefunc: Callable[[], Any]*)

Construct a new **async_scoped_session**.

Parameters:

- session_factory** – a factory to create new **AsyncSession** instances. This is usually, but not necessarily, an instance of **async_sessionmaker**.
- scopefunc** – function which defines the current scope. A function such as `asyncio.current_task` may be useful here.

method **sqlalchemy.ext.asyncio.async_scoped_session.async aclose**() → *None*

A synonym for **AsyncSession.close()**.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

The **AsyncSession.aclose()** name is specifically to support the Python standard library `@contextlib.aclosing` context manager function.

Added in version 2.0.20.

method **sqlalchemy.ext.asyncio.async_scoped_session.add**(*instance: object, *, _warn: bool = True*) → *None*

Place an object into this **Session**.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

Objects that are in the **transient** state when passed to the **Session.add()** method will move to the **pending** state, until the next flush, at which point they will move to the **persistent** state.

Objects that are in the **detached** state when passed to the **Session.add()** method will move to the **persistent** state directly.

If the transaction used by the **Session** is rolled back, objects which were transient when they were passed to **Session.add()** will be moved back to the **transient** state, and will no longer be present within this **Session**.

See also

Session.add_all()

Adding New or Existing Items - at Basics of Using a Session

method **sqlalchemy.ext.asyncio.async_scoped_session.add_all**(*instances: Iterable[object]*) → *None*

Add the given collection of instances to this **Session**.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

See the documentation for `Session.add()` for a general behavioral description.

See also

`Session.add()`

Adding New or Existing Items - at Basics of Using a Session

property `autoflush`: Any

Proxy for the `Session.autoflush` attribute on behalf of the `AsyncSession` class.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

method `sqlalchemy.ext.asyncio.async_scoped_session.begin()` → `AsyncSessionTransaction`

Return an `AsyncSessionTransaction` object.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

The underlying `Session` will perform the “begin” action when the `AsyncSessionTransaction` object is entered:

```
async with async_session.begin():
    ... # ORM transaction is begun
```

Note that database IO will not normally occur when the session-level transaction is begun, as database transactions begin on an on-demand basis. However, the begin block is async to accommodate for a `SessionEvents.after_transaction_create()` event hook that may perform IO.

For a general description of ORM begin, see `Session.begin()`.

method `sqlalchemy.ext.asyncio.async_scoped_session.begin_nested()` → `AsyncSessionTransaction`

Return an `AsyncSessionTransaction` object which will begin a “nested” transaction, e.g. SAVEPOINT.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Behavior is the same as that of `AsyncSession.begin()`.

For a general description of ORM begin nested, see `Session.begin_nested()`.

See also

Serializable isolation / Savepoints / Transactional DDL (asyncio version) - special workarounds required with the SQLite asyncio driver in order for SAVEPOINT to work correctly.

property `bind`: Any

Proxy for the `AsyncSession.bind` attribute on behalf of the `async_scoped_session` class.

method `sqlalchemy.ext.asyncio.async_scoped_session.async close()` → None

Close out the transactional resources and ORM objects used by this `AsyncSession`.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

See also

`Session.close()` - main documentation for “close”

Closing - detail on the semantics of `AsyncSession.close()` and `AsyncSession.reset()`.

async classmethod `sqlalchemy.ext.asyncio.async_scoped_session.close_all()` → None

Close all `AsyncSession` sessions.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Deprecated since version 2.0: The `AsyncSession.close_all()` method is deprecated and will be removed in a future release. Please refer to `close_all_sessions()`.

method `sqlalchemy.ext.asyncio.async_scoped_session.async commit()` → None

Commit the current transaction in progress.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

See also

`Session.commit()` - main documentation for “commit”

method `sqlalchemy.ext.asyncio.async_scoped_session.configure(**kwargs: Any)` → None

reconfigure the `sessionmaker` used by this `scoped_session`.

See `sessionmaker.configure()`.

method `sqlalchemy.ext.asyncio.async_scoped_session.async connection(bind_arguments: _BindArguments | None = None, execution_options: CoreExecuteOptionsParameter | None = None, **kw: Any)` → `AsyncConnection`

Return a `AsyncConnection` object corresponding to this `Session` object’s transactional state.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

This method may also be used to establish execution options for the database connection used by the current transaction.

Added in version 1.4.24: Added `**kw` arguments which are passed through to the underlying `Session.connection()` method.

See also

`Session.connection()` - main documentation for “connection”

method `sqlalchemy.ext.asyncio.async_scoped_session.async delete(instance: object)` → None

Mark an instance as deleted.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

The database delete operation occurs upon `flush()`.

As this operation may need to cascade along unloaded relationships, it is awaitable to allow for those queries to take place.

See also

`Session.delete()` - main documentation for delete

method `sqlalchemy.ext.asyncio.async_scoped_session.async delete_all(instances: Iterable[object])` → None

Calls `AsyncSession.delete()` on multiple instances.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

See also

`Session.delete_all()` - main documentation for delete_all

property `deleted`: Any

The set of all instances marked as ‘deleted’ within this `Session`

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

property `dirty`: Any

The set of all persistent instances considered dirty.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

E.g.:

```
some_mapped_object in session.dirty
```

Instances are considered dirty when they were modified but not deleted.

Note that this ‘dirty’ calculation is ‘optimistic’; most attribute-setting or collection modification operations will mark an instance as ‘dirty’ and place it in this set, even if there is no net change to the attribute’s value. At flush time, the value of each attribute is compared to its previously saved value, and if there’s no net change, no SQL operation will occur (this is a more expensive operation so it’s only done at flush time).

To check if an instance has actionable net changes to its attributes, use the **Session.is_modified()** method.

method **sqlalchemy.ext.asyncio.async_scoped_session.async execute**(statement: *Executable*, params: *_CoreAnyExecuteParams* / *None* = *None*,**, execution_options: OrmExecuteOptionsParameter* = {}, bind_arguments: *_BindArguments* / *None* = *None*, ***kw: Any*) → **Result**[Unpack[TupleAny]]

Execute a statement and return a buffered **Result** object.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

See also

Session.execute() - main documentation for execute

property **execution_options: Any**

Proxy for the **Session.execution_options** attribute on behalf of the **AsyncSession** class.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

method **sqlalchemy.ext.asyncio.async_scoped_session.expire**(instance: *object*, attribute_names: *Iterable[str]* / *None* = *None*) → *None*

Expire the attributes on an instance.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

Marks the attributes of an instance as out of date. When an expired attribute is next accessed, a query will be issued to the **Session** object’s current transactional context in order to load all expired attributes for the given instance. Note that a highly isolated transaction will return the same values as were previously read in that same transaction, regardless of changes in database state outside of that transaction.

To expire all objects in the **Session** simultaneously, use **Session.expire_all()**.

The **Session** object’s default behavior is to expire all state whenever the **Session.rollback()** or **Session.commit()** methods are called, so that new state can be loaded for the new transaction. For this reason, calling **Session.expire()** only makes sense for the specific case that a non-ORM SQL statement was emitted in the current transaction.

Parameters:

- **instance** – The instance to be refreshed.
- **attribute_names** – optional list of string attribute names indicating a subset of attributes to be expired.

See also

Refreshing / Expiring - introductory material

Session.expire()

Session.refresh()

Query.populate_existing()

method **sqlalchemy.ext.asyncio.async_scoped_session.expire_all**() → *None*

Expires all persistent instances within this Session.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

When any attributes on a persistent instance is next accessed, a query will be issued using the **Session** object’s current transactional context in order to load all expired attributes for the given instance. Note that a highly isolated transaction will return the same values as were previously read in that same transaction, regardless of changes in database state outside of that transaction.

To expire individual objects and individual attributes on those objects, use `Session.expire()`.

The **Session** object’s default behavior is to expire all state whenever the `Session.rollback()` or `Session.commit()` methods are called, so that new state can be loaded for the new transaction. For this reason, calling `Session.expire_all()` is not usually needed, assuming the transaction is isolated.

See also

[Refreshing / Expiring](#) - introductory material

Session.expire()

Session.refresh()

Query.populate_existing()

method **sqlalchemy.ext.asyncio.async_scoped_session.expunge**(*instance: **Object***) → None

Remove the *instance* from this *Session*.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

This will free all internal references to the instance. Cascading will be applied according to the *expunge* cascade rule.

method **sqlalchemy.ext.asyncio.async_scoped_session.expunge_all**() → None

Remove all object instances from this *Session*.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

This is equivalent to calling `expunge(obj)` on all objects in this *Session*.

method **sqlalchemy.ext.asyncio.async_scoped_session.async flush**(*objects: Sequence[Any] | None = None*) → None

Flush all the object changes to the database.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

See also

Session.flush() - main documentation for flush

method **sqlalchemy.ext.asyncio.async_scoped_session.async get**(*entity: _EntityBindKey[_O], ident: _PKIdentityArgument, *, options: Sequence[ORMOption] | None = None, populate_existing: bool = False, with_for_update: ForUpdateParameter = None, identity_token: Any | None = None, execution_options: OrmExecuteOptionsParameter = {}*) → *_O* | None

Return an instance based on the given primary key identifier, or *None* if not found.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

See also

Session.get() - main documentation for get

method `sqlalchemy.ext.asyncio.async_scoped_session.get_bind`(`mapper: _EntityBindKey[_O] | None = None`, `clause: ClauseElement | None = None`, `bind: _SessionBind | None = None`, ***kw: Any*) → `Engine | Connection`

Return a “bind” to which the synchronous proxied `Session` is bound.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Unlike the `Session.get_bind()` method, this method is currently **not** used by this `AsyncSession` in any way in order to resolve engines for requests.

Note

This method proxies directly to the `Session.get_bind()` method, however is currently **not** useful as an override target, in contrast to that of the `Session.get_bind()` method. The example below illustrates how to implement custom `Session.get_bind()` schemes that work with `AsyncSession` and `AsyncEngine`.

The pattern introduced at [Custom Vertical Partitioning](#) illustrates how to apply a custom bind-lookup scheme to a `Session` given a set of `Engine` objects. To apply a corresponding `Session.get_bind()` implementation for use with a `AsyncSession` and `AsyncEngine` objects, continue to subclass `Session` and apply it to `AsyncSession` using `AsyncSession.sync_session_class`. The inner method must continue to return `Engine` instances, which can be acquired from a `AsyncEngine` using the `AsyncEngine.sync_engine` attribute:

```
# using example from "Custom Vertical Partitioning"

import random

from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.ext.asyncio import create_async_engine
from sqlalchemy.ext.asyncio import async_sessionmaker
from sqlalchemy.orm import Session

# construct async engines w/ async drivers
engines = {
    "leader": create_async_engine("sqlite+aiosqlite:///leader.db"),
    "other": create_async_engine("sqlite+aiosqlite:///other.db"),
    "follower1": create_async_engine("sqlite+aiosqlite:///follower1.db"),
    "follower2": create_async_engine("sqlite+aiosqlite:///follower2.db"),
}

class RoutingSession(Session):
    def get_bind(self, mapper=None, clause=None, **kw):
        # within get_bind(), return sync engines
        if mapper and issubclass(mapper.class_, MyOtherClass):
            return engines["other"].sync_engine
        elif self._flushing or isinstance(clause, (Update, Delete)):
            return engines["leader"].sync_engine
        else:
            return engines[
                random.choice(["follower1", "follower2"])
            ].sync_engine

# apply to AsyncSession using sync_session_class
AsyncSessionMaker = async_sessionmaker(sync_session_class=RoutingSession)
```

Non-Annotated Example

The `Session.get_bind()` method is called in a non-asyncio, implicitly non-blocking context in the same manner as ORM event hooks and functions that are invoked via `AsyncSession.run_sync()`, so routines that wish to run SQL commands inside of `Session.get_bind()` can continue to do so using blocking-style code, which will be translated to implicitly async calls at the point of invoking IO on the database drivers.

method `sqlalchemy.ext.asyncio.async_scoped_session.async_get_one`(`entity: _EntityBindKey[_O]`, `ident: _PKIdentityArgument, *, options: Sequence[ORMOption] | None = None`, `populate_existing: bool = False`, `with_for_update: ForUpdateParameter = None`, `identity_token: Any | None = None`, `execution_options: OrmExecuteOptionsParameter = {}`) → `_O`

Return an instance based on the given primary key identifier, or raise an exception if not found.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Raises `NoResultFound` if the query selects no rows.

..versionadded: 2.0.22

See also

`Session.get_one()` - main documentation for `get_one`

classmethod `sqlalchemy.ext.asyncio.async_scoped_session.identity_key`(*class_*: *Type*[*Any*] | *None* = *None*, *ident*: *Any* | *Tuple*[*Any*, ...] = *None*,^{*} *instance*: *Any* | *None* = *None*, *row*: *Row*[*Unpack*[*TupleAny*]] | *RowMapping* | *None* = *None*, *identity_token*: *Any* | *None* = *None*) → *_IdentityKeyType*[*Any*]

Return an identity key.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

This is an alias of `identity_key()`.

property `identity_map`: Any

Proxy for the `Session.identity_map` attribute on behalf of the `AsyncSession` class.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

property `info`: Any

A user-modifiable dictionary.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

The initial value of this dictionary can be populated using the `info` argument to the `Session` constructor or `sessionmaker` constructor or factory methods. The dictionary here is always local to this `Session` and can be modified independently of all other `Session` objects.

method `sqlalchemy.ext.asyncio.async_scoped_session.async invalidate`() → *None*

Close this Session, using connection invalidation.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

For a complete description, see `Session.invalidate()`.

property `is_active`: Any

True if this `Session` not in “partial rollback” state.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

Changed in version 1.4: The `Session` no longer begins a new transaction immediately, so this attribute will be False when the `Session` is first instantiated.

“partial rollback” state typically indicates that the flush process of the `Session` has failed, and that the `Session.rollback()` method must be emitted in order to fully roll back the transaction.

If this `Session` is not in a transaction at all, the `Session` will autobegin when it is first used, so in this case `Session.is_active` will return True.

Otherwise, if this `Session` is within a transaction, and that transaction has not been rolled back internally, the `Session.is_active` will also return True.

See also

“This Session’s transaction has been rolled back due to a previous exception during flush.” (or similar)

`Session.in_transaction()`

method `sqlalchemy.ext.asyncio.async_scoped_session.is_modified`(*instance*: *object*, *include_collections*: *bool* = *True*) → *bool*

Return `True` if the given instance has locally modified attributes.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

This method retrieves the history for each instrumented attribute on the instance and performs a comparison of the current value to its previously flushed or committed value, if any.

It is in effect a more expensive and accurate version of checking for the given instance in the `Session.dirty` collection; a full test for each attribute’s net “dirty” status is performed.

E.g.:

```
return session.is_modified(someobject)
```

A few caveats to this method apply:

- Instances present in the `Session.dirty` collection may report `False` when tested with this method. This is because the object may have received change events via attribute mutation, thus placing it in `Session.dirty`, but ultimately the state is the same as that loaded from the database, resulting in no net change here.
- Scalar attributes may not have recorded the previously set value when a new value was applied, if the attribute was not loaded, or was expired, at the time the new value was received - in these cases, the attribute is assumed to have a change, even if there is ultimately no net change against its database value. SQLAlchemy in most cases does not need the “old” value when a set event occurs, so it skips the expense of a SQL call if the old value isn’t present, based on the assumption that an UPDATE of the scalar value is usually needed, and in those few cases where it isn’t, is less expensive on average than issuing a defensive SELECT.

The “old” value is fetched unconditionally upon set only if the attribute container has the `active_history` flag set to `True`. This flag is set typically for primary key attributes and scalar object references that are not a simple many-to-one. To set this flag for any arbitrary mapped column, use the `active_history` argument with `column_property()`.

Parameters:

- instance** – mapped instance to be tested for pending changes.
- include_collections** – Indicates if multivalued collections should be included in the operation. Setting this to `False` is a way to detect only local-column based properties (i.e. scalar columns or many-to-one foreign keys) that would result in an UPDATE for this instance upon flush.

method `sqlalchemy.ext.asyncio.async_scoped_session.async merge(instance: _O, *, load: bool = True, options: Sequence[ORMOption] | None = None) → _O`

Copy the state of a given instance into a corresponding instance within this `AsyncSession`.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

See also

`Session.merge()` - main documentation for merge

method `sqlalchemy.ext.asyncio.async_scoped_session.async merge_all(instances: Iterable[_O], *, load: bool = True, options: Sequence[ORMOption] | None = None) → Sequence[_O]`

Calls `AsyncSession.merge()` on multiple instances.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

See also

`Session.merge_all()` - main documentation for merge_all

property `new`: Any

The set of all instances marked as ‘new’ within this `Session`.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

property `no_autoflush`: Any

Return a context manager that disables autoflush.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

e.g.:

```
with session.no_autoflush:

    some_object = SomeClass()
    session.add(some_object)
    # won't autoflush
    some_object.related_thing = session.query(SomeRelated).first()
```

Operations that proceed within the `with`: block will not be subject to flushes occurring upon query access. This is useful when initializing a series of objects which involve existing database queries, where the uncompleted object should not yet be flushed.

classmethod `sqlalchemy.ext.asyncio.async_scoped_session.object_session`(instance: *object*) → `Session` | `None`

Return the `Session` to which an object belongs.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

This is an alias of `object_session()`.

method `sqlalchemy.ext.asyncio.async_scoped_session.async refresh`(instance: *object*, attribute_names: *Iterable[str]* | *None* = *None*, with_for_update: *ForUpdateParameter* = *None*) → `None`

Expire and refresh the attributes on the given instance.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

A query will be issued to the database and all attributes will be refreshed with their current database value.

This is the async version of the `Session.refresh()` method. See that method for a complete description of all options.

See also

`Session.refresh()` - main documentation for refresh

method `sqlalchemy.ext.asyncio.async_scoped_session.async remove`() → `None`

Dispose of the current `AsyncSession`, if present.

Different from `scoped_session`’s `remove` method, this method would use `await` to wait for the `close` method of `AsyncSession`.

method `sqlalchemy.ext.asyncio.async_scoped_session.async reset`() → `None`

Close out the transactional resources and ORM objects used by this `Session`, resetting the session to its initial state.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

Added in version 2.0.22.

See also

`Session.reset()` - main documentation for “reset”

Closing - detail on the semantics of `AsyncSession.close()` and `AsyncSession.reset()`.

method `sqlalchemy.ext.asyncio.async_scoped_session.async rollback`() → `None`

Rollback the current transaction in progress.

Proxied for the `AsyncSession` class on behalf of the `async_scoped_session` class.

See also

`Session.rollback()` - main documentation for “rollback”

method `sqlalchemy.ext.asyncio.async_scoped_session.async scalar`(statement: *Executable*, params: *_CoreAnyExecuteParams* | *None* = *None*, *, execution_options: *OrmExecuteOptionsParameter* = {}, bind_arguments: *_BindArguments* | *None* = *None*, **kw: *Any*) → *Any*

Execute a statement and return a scalar result.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

See also

Session.scalar() - main documentation for scalar

method **sqlalchemy.ext.asyncio.async_scoped_session**.async **scalars**(statement: *Executable*, params: *_CoreAnyExecuteParams* / *None* = *None*,*, execution_options: *OrmExecuteOptionsParameter* = {}, bind_arguments: *_BindArguments* / *None* = *None*, **kw: Any) → **ScalarResult**[Any]

Execute a statement and return scalar results.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

Returns:

a **ScalarResult** object

Added in version 1.4.24: Added **AsyncSession.scalars()**

Added in version 1.4.26: Added **async_scoped_session.scalars()**

See also

Session.scalars() - main documentation for scalars

AsyncSession.stream_scalars() - streaming version

attribute **sqlalchemy.ext.asyncio.async_scoped_session.session_factory**: *async_sessionmaker*[_AS]

The *session_factory* provided to *__init__* is stored in this attribute and may be accessed at a later time. This can be useful when a new non-scoped **AsyncSession** is needed.

method **sqlalchemy.ext.asyncio.async_scoped_session**.async **stream**(statement: *Executable*, params: *_CoreAnyExecuteParams* / *None* = *None*,*, execution_options: *OrmExecuteOptionsParameter* = {}, bind_arguments: *_BindArguments* / *None* = *None*, **kw: Any) → **AsyncResult**[Unpack[TupleAny]]

Execute a statement and return a streaming **AsyncResult** object.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

method **sqlalchemy.ext.asyncio.async_scoped_session**.async **stream_scalars**(statement: *Executable*, params: *_CoreAnyExecuteParams* / *None* = *None*,*, execution_options: *OrmExecuteOptionsParameter* = {}, bind_arguments: *_BindArguments* / *None* = *None*, **kw: Any) → **AsyncScalarResult**[Any]

Execute a statement and return a stream of scalar results.

Proxied for the **AsyncSession** class on behalf of the **async_scoped_session** class.

Returns:

an **AsyncScalarResult** object

Added in version 1.4.24.

See also

Session.scalars() - main documentation for scalars

AsyncSession.scalars() - non streaming version

class **sqlalchemy.ext.asyncio.AsyncAttrs**

Mixin class which provides an awaitable accessor for all attributes.

E.g.:

```
from __future__ import annotations

from typing import List

from sqlalchemy import ForeignKey
from sqlalchemy import func
from sqlalchemy.ext.asyncio import AsyncAttrs
from sqlalchemy.orm import DeclarativeBase
from sqlalchemy.orm import Mapped
from sqlalchemy.orm import mapped_column
from sqlalchemy.orm import relationship


class Base(AsyncAttrs, DeclarativeBase):
    pass


class A(Base):
    __tablename__ = "a"

    id: Mapped[int] = mapped_column(primary_key=True)
    data: Mapped[str]
    bs: Mapped[List[B]] = relationship()


class B(Base):
    __tablename__ = "b"
    id: Mapped[int] = mapped_column(primary_key=True)
    a_id: Mapped[int] = mapped_column(ForeignKey("a.id"))
    data: Mapped[str]
```

Annotated Example

In the above example, the **AsyncAttrs** mixin is applied to the declarative **Base** class where it takes effect for all subclasses. This mixin adds a single new attribute **AsyncAttrs.awaitable_attrs** to all classes, which will yield the value of any attribute as an awaitable. This allows attributes which may be subject to lazy loading or deferred / unexpiry loading to be accessed such that IO can still be emitted:

```
a1 = (await async_session.scalars(select(A).where(A.id == 5))).one()

# use the lazy loader on ``a1.bs`` via the ``.awaitable_attrs``
# interface, so that it may be awaited
for b1 in await a1.awaitable_attrs.bs:
    print(b1)
```

The **AsyncAttrs.awaitable_attrs** performs a call against the attribute that is approximately equivalent to using the **AsyncSession.run_sync()** method, e.g.:

```
for b1 in await async_session.run_sync(lambda sess: a1.bs):
    print(b1)
```

Added in version 2.0.13.

See also

[Preventing Implicit IO when Using AsyncSession](#)

property **awaitable_attrs**: **_AsyncAttrGetitem**

provide a namespace of all attributes on this object wrapped as awaitables.

e.g.:

```
a1 = (await async_session.scalars(select(A).where(A.id == 5))).one()

some_attribute = await a1.awaitable_attrs.some_deferred_attribute
some_collection = await a1.awaitable_attrs.some_collection
```

class sqlalchemy.ext.asyncio.**AsyncSession**

inherits from sqlalchemy.ext.asyncio.base.ReversibleProxy

Asyncio version of **Session**.

The **AsyncSession** is a proxy for a traditional **Session** instance.

The **AsyncSession** is **not safe for use in concurrent tasks**.. See [Is the Session thread-safe? Is AsyncSession safe to share in concurrent tasks?](#) for background.

Added in version 1.4.

To use an **AsyncSession** with custom **Session** implementations, see the **AsyncSession.sync_session_class** parameter.

Member Name	Description
<code>__init__()</code>	Construct a new AsyncSession .
<code>aclose()</code>	A synonym for AsyncSession.close() .
<code>add()</code>	Place an object into this Session .
<code>add_all()</code>	Add the given collection of instances to this Session .
<code>begin()</code>	Return an AsyncSessionTransaction object.
<code>begin_nested()</code>	Return an AsyncSessionTransaction object which will begin a “nested” transaction, e.g. SAVEPOINT.
<code>close()</code>	Close out the transactional resources and ORM objects used by this AsyncSession .
<code>close_all()</code>	Close all AsyncSession sessions.
<code>commit()</code>	Commit the current transaction in progress.
<code>connection()</code>	Return a AsyncConnection object corresponding to this Session object’s transactional state.
<code>delete()</code>	Mark an instance as deleted.
<code>delete_all()</code>	Calls AsyncSession.delete() on multiple instances.
<code>execute()</code>	Execute a statement and return a buffered Result object.
<code>expire()</code>	Expire the attributes on an instance.
<code>expire_all()</code>	Expires all persistent instances within this Session.
<code>expunge()</code>	Remove the <i>instance</i> from this Session .
<code>expunge_all()</code>	Remove all object instances from this Session .
<code>flush()</code>	Flush all the object changes to the database.
<code>get()</code>	Return an instance based on the given primary key identifier, or None if not found.
<code>get_bind()</code>	Return a “bind” to which the synchronous proxied Session is bound.
<code>get_nested_transaction()</code>	Return the current nested transaction in progress, if any.
<code>get_one()</code>	Return an instance based on the given primary key identifier, or raise an exception if not found.
<code>get_transaction()</code>	Return the current root transaction in progress, if any.
<code>identity_key()</code>	Return an identity key.
<code>in_nested_transaction()</code>	Return True if this Session has begun a nested transaction, e.g. SAVEPOINT.
<code>in_transaction()</code>	Return True if this Session has begun a transaction.
<code>invalidate()</code>	Close this Session, using connection invalidation.
<code>is_modified()</code>	Return True if the given instance has locally modified attributes.
<code>merge()</code>	Copy the state of a given instance into a corresponding instance within this AsyncSession .
<code>merge_all()</code>	Calls AsyncSession.merge() on multiple instances.
<code>object_session()</code>	Return the Session to which an object belongs.
<code>refresh()</code>	Expire and refresh the attributes on the given instance.
<code>reset()</code>	Close out the transactional resources and ORM objects used by this Session , resetting the session to its initial state.
<code>rollback()</code>	Rollback the current transaction in progress.
<code>run_sync()</code>	Invoke the given synchronous (i.e. not async) callable, passing a synchronous-style Session as the first argument.
<code>scalar()</code>	Execute a statement and return a scalar result.

scalars()	Execute a statement and return scalar results.
stream()	Execute a statement and return a streaming AsyncResult object.
stream_scalars()	Execute a statement and return a stream of scalar results.
sync_session	Reference to the underlying Session this AsyncSession proxies requests towards.
sync_session_class	The class or callable that provides the underlying Session instance for a particular AsyncSession .

method **sqlalchemy.ext.asyncio.AsyncSession.__init__**(*bind: _AsyncSessionBind | None = None,* binds: Dict[_SessionBindKey, _AsyncSessionBind] | None = None, sync_session_class: Type[Session] | None = None, **kw: Any*)

Construct a new **AsyncSession**.

All parameters other than `sync_session_class` are passed to the `sync_session_class` callable directly to instantiate a new **Session**. Refer to **Session.__init__()** for parameter documentation.

Parameters:

sync_session_class –

A **Session** subclass or other callable which will be used to construct the **Session** which will be proxied. This parameter may be used to provide custom **Session** subclasses. Defaults to the **AsyncSession.sync_session_class** class-level attribute.

Added in version 1.4.24.

method **sqlalchemy.ext.asyncio.AsyncSession.async aclose()** → None

A synonym for **AsyncSession.close()**.

The **AsyncSession.aclose()** name is specifically to support the Python standard library `@contextlib.aclosing` context manager function.

Added in version 2.0.20.

method **sqlalchemy.ext.asyncio.AsyncSession.add**(*instance: object,* _warn: bool = True*) → None

Place an object into this **Session**.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

Objects that are in the **transient** state when passed to the **Session.add()** method will move to the **pending** state, until the next flush, at which point they will move to the **persistent** state.

Objects that are in the **detached** state when passed to the **Session.add()** method will move to the **persistent** state directly.

If the transaction used by the **Session** is rolled back, objects which were transient when they were passed to **Session.add()** will be moved back to the **transient** state, and will no longer be present within this **Session**.

See also

Session.add_all()

Adding New or Existing Items - at Basics of Using a Session

method **sqlalchemy.ext.asyncio.AsyncSession.add_all**(*instances: Iterable[object]*) → None

Add the given collection of instances to this **Session**.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

See the documentation for **Session.add()** for a general behavioral description.

See also

Session.add()

Adding New or Existing Items - at Basics of Using a Session

property **autoflush**: bool

Proxy for the `Session.autoflush` attribute on behalf of the `AsyncSession` class.

method `sqlalchemy.ext.asyncio.AsyncSession.begin()` → `AsyncSessionTransaction`

Return an `AsyncSessionTransaction` object.

The underlying `Session` will perform the “begin” action when the `AsyncSessionTransaction` object is entered:

```
async with async_session.begin():
    ... # ORM transaction is begun
```

Note that database IO will not normally occur when the session-level transaction is begun, as database transactions begin on an on-demand basis. However, the begin block is async to accommodate for a `SessionEvents.after_transaction_create()` event hook that may perform IO.

For a general description of ORM begin, see `Session.begin()`.

method `sqlalchemy.ext.asyncio.AsyncSession.begin_nested()` → `AsyncSessionTransaction`

Return an `AsyncSessionTransaction` object which will begin a “nested” transaction, e.g. SAVEPOINT.

Behavior is the same as that of `AsyncSession.begin()`.

For a general description of ORM begin nested, see `Session.begin_nested()`.

See also

`Serializable isolation / Savepoints / Transactional DDL (asyncio version)` - special workarounds required with the SQLite asyncio driver in order for SAVEPOINT to work correctly.

method `sqlalchemy.ext.asyncio.AsyncSession.async close()` → `None`

Close out the transactional resources and ORM objects used by this `AsyncSession`.

See also

`Session.close()` - main documentation for “close”

`Closing` - detail on the semantics of `AsyncSession.close()` and `AsyncSession.reset()`.

async classmethod `sqlalchemy.ext.asyncio.AsyncSession.close_all()` → `None`

Close all `AsyncSession` sessions.

Deprecated since version 2.0: The `AsyncSession.close_all()` method is deprecated and will be removed in a future release. Please refer to `close_all_sessions()`.

method `sqlalchemy.ext.asyncio.AsyncSession.async commit()` → `None`

Commit the current transaction in progress.

See also

`Session.commit()` - main documentation for “commit”

method `sqlalchemy.ext.asyncio.AsyncSession.async connection(bind_arguments: _BindArguments | None = None, execution_options: CoreExecuteOptionsParameter | None = None, **kw: Any)` → `AsyncConnection`

Return a `AsyncConnection` object corresponding to this `Session` object’s transactional state.

This method may also be used to establish execution options for the database connection used by the current transaction.

Added in version 1.4.24: Added `**kw` arguments which are passed through to the underlying `Session.connection()` method.

See also

`Session.connection()` - main documentation for “connection”

method `sqlalchemy.ext.asyncio.AsyncSession.async delete(instance: object)` → `None`

Mark an instance as deleted.

The database delete operation occurs upon `flush()`.

As this operation may need to cascade along unloaded relationships, it is awaitable to allow for those queries to take place.

See also

`Session.delete()` - main documentation for delete

method `sqlalchemy.ext.asyncio.AsyncSession`.async **delete_all**(*instances: Iterable[object]*) → None

Calls `AsyncSession.delete()` on multiple instances.

See also

`Session.delete_all()` - main documentation for delete_all

property **deleted**: Any

The set of all instances marked as ‘deleted’ within this `Session`

Proxied for the `Session` class on behalf of the `AsyncSession` class.

property **dirty**: Any

The set of all persistent instances considered dirty.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

E.g.:

```
some_mapped_object in session.dirty
```

Instances are considered dirty when they were modified but not deleted.

Note that this ‘dirty’ calculation is ‘optimistic’; most attribute-setting or collection modification operations will mark an instance as ‘dirty’ and place it in this set, even if there is no net change to the attribute’s value. At flush time, the value of each attribute is compared to its previously saved value, and if there’s no net change, no SQL operation will occur (this is a more expensive operation so it’s only done at flush time).

To check if an instance has actionable net changes to its attributes, use the `Session.is_modified()` method.

method `sqlalchemy.ext.asyncio.AsyncSession`.async **execute**(*statement: Executable, params: _CoreAnyExecuteParams | None = None, ...*, execution_options: OrmExecuteOptionsParameter = {}, bind_arguments: _BindArguments | None = None, **kw: Any*) → `Result[Unpack[TupleAny]]`

Execute a statement and return a buffered `Result` object.

See also

`Session.execute()` - main documentation for execute

property **execution_options**: `ExecuteOptions`

Proxy for the `Session.execution_options` attribute on behalf of the `AsyncSession` class.

method `sqlalchemy.ext.asyncio.AsyncSession`. **expire**(*instance: object, attribute_names: Iterable[str] | None = None*) → None

Expire the attributes on an instance.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

Marks the attributes of an instance as out of date. When an expired attribute is next accessed, a query will be issued to the `Session` object’s current transactional context in order to load all expired attributes for the given instance. Note that a highly isolated transaction will return the same values as were previously read in that same transaction, regardless of changes in database state outside of that transaction.

To expire all objects in the `Session` simultaneously, use `Session.expire_all()`.

The `Session` object’s default behavior is to expire all state whenever the `Session.rollback()` or `Session.commit()` methods are called, so that new state can be loaded for the new transaction. For this reason, calling `Session.expire()` only makes sense for the specific case that a non-ORM SQL statement was emitted in the current transaction.

Parameters:

- instance** – The instance to be refreshed.
- attribute_names** – optional list of string attribute names indicating a subset of attributes to be expired.

See also

[Refreshing / Expiring](#) - introductory material

`Session.expire()`

`Session.refresh()`

`Query.populate_existing()`

method `sqlalchemy.ext.asyncio.AsyncSession.expire_all()` → None

Expires all persistent instances within this `Session`.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

When any attributes on a persistent instance is next accessed, a query will be issued using the `Session` object’s current transactional context in order to load all expired attributes for the given instance. Note that a highly isolated transaction will return the same values as were previously read in that same transaction, regardless of changes in database state outside of that transaction.

To expire individual objects and individual attributes on those objects, use `Session.expire()`.

The `Session` object’s default behavior is to expire all state whenever the `Session.rollback()` or `Session.commit()` methods are called, so that new state can be loaded for the new transaction. For this reason, calling `Session.expire_all()` is not usually needed, assuming the transaction is isolated.

See also

[Refreshing / Expiring](#) - introductory material

`Session.expire()`

`Session.refresh()`

`Query.populate_existing()`

method `sqlalchemy.ext.asyncio.AsyncSession.expunge(instance: object)` → None

Remove the *instance* from this `Session`.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

This will free all internal references to the instance. Cascading will be applied according to the *expunge* cascade rule.

method `sqlalchemy.ext.asyncio.AsyncSession.expunge_all()` → None

Remove all object instances from this `Session`.

Proxied for the `Session` class on behalf of the `AsyncSession` class.

This is equivalent to calling `expunge(obj)` on all objects in this `Session`.

method `sqlalchemy.ext.asyncio.AsyncSession.async flush(objects: Sequence[Any] | None = None)` → None

Flush all the object changes to the database.

See also

`Session.flush()` - main documentation for flush

method `sqlalchemy.ext.asyncio.AsyncSession.async get(entity: _EntityBindKey[_O], idenfity: _PKIdentityArgument, *, options: Sequence[ORMOption] | None = None, populate_existing: bool = False, with_for_update: ForUpdateParameter = None, identity_token: Any | None = None, execution_options: OrmExecuteOptionsParameter = {})` → `_O` | None

Return an instance based on the given primary key identifier, or `None` if not found.

See also

`Session.get()` - main documentation for get

method `sqlalchemy.ext.asyncio.AsyncSession.get_bind(mapper: _EntityBindKey[_O] | None = None, clause: ClauseElement | None = None, bind: _SessionBind | None = None, **kw: Any)` → `Engine` | `Connection`

Return a “bind” to which the synchronous proxied `Session` is bound.

Unlike the `Session.get_bind()` method, this method is currently **not** used by this `AsyncSession` in any way in order to resolve engines for requests.

Note

This method proxies directly to the `Session.get_bind()` method, however is currently **not** useful as an override target, in contrast to that of the `Session.get_bind()` method. The example below illustrates how to implement custom `Session.get_bind()` schemes that work with `AsyncSession` and `AsyncEngine`.

The pattern introduced at [Custom Vertical Partitioning](#) illustrates how to apply a custom bind-lookup scheme to a `Session` given a set of `Engine` objects. To apply a corresponding `Session.get_bind()` implementation for use with a `AsyncSession` and `AsyncEngine` objects, continue to subclass `Session` and apply it to `AsyncSession` using `AsyncSession.sync_session_class`. The inner method must continue to return `Engine` instances, which can be acquired from a `AsyncEngine` using the `AsyncEngine.sync_engine` attribute:

```
# using example from "Custom Vertical Partitioning"

import random

from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.ext.asyncio import create_async_engine
from sqlalchemy.ext.asyncio import async_sessionmaker
from sqlalchemy.orm import Session

# construct async engines w/ async drivers
engines = {
    "leader": create_async_engine("sqlite+aiosqlite:///leader.db"),
    "other": create_async_engine("sqlite+aiosqlite:///other.db"),
    "follower1": create_async_engine("sqlite+aiosqlite:///follower1.db"),
    "follower2": create_async_engine("sqlite+aiosqlite:///follower2.db"),
}

class RoutingSession(Session):
    def get_bind(self, mapper=None, clause=None, **kw):
        # within get_bind(), return sync engines
        if mapper and issubclass(mapper.class_, MyOtherClass):
            return engines["other"].sync_engine
        elif self._flushing or isinstance(clause, (Update, Delete)):
            return engines["leader"].sync_engine
        else:
            return engines[
                random.choice(["follower1", "follower2"])
            ].sync_engine

# apply to AsyncSession using sync_session_class
AsyncSessionMaker = async_sessionmaker(sync_session_class=RoutingSession)
```

Non-Annotated Example

The `Session.get_bind()` method is called in a non-asyncio, implicitly non-blocking context in the same manner as ORM event hooks and functions that are invoked via `AsyncSession.run_sync()`, so routines that wish to run SQL commands inside of `Session.get_bind()` can continue to do so using blocking-style code, which will be translated to implicitly async calls at the point of invoking IO on the database drivers.

method `sqlalchemy.ext.asyncio.AsyncSession.get_nested_transaction()` → `AsyncSessionTransaction` | `None`

Return the current nested transaction in progress, if any.

Returns:

an `AsyncSessionTransaction` object, or `None`.

Added in version 1.4.18.

method `sqlalchemy.ext.asyncio.AsyncSession.async get_one(entity: _EntityBindKey[_O], ident: _PKIdentityArgument, *, options: Sequence[ORMOption] | None = None, populate_existing: bool = False, with_for_update: ForUpdateParameter = None, identity_token: Any | None = None, execution_options: OrmExecuteOptionsParameter = {})` → `_O`

Return an instance based on the given primary key identifier, or raise an exception if not found.

Raises `NoResultFound` if the query selects no rows.

..versionadded: 2.0.22

See also

`Session.get_one()` - main documentation for `get_one`

method `sqlalchemy.ext.asyncio.AsyncSession.get_transaction()` → `AsyncSessionTransaction` | `None`

Return the current root transaction in progress, if any.

Returns:

an **AsyncSessionTransaction** object, or `None`.

Added in version 1.4.18.

classmethod **sqlalchemy.ext.asyncio.AsyncSession.identity_key**(*class_*: *Type[Any] | None = None*, *ident*: *Any | Tuple[Any, ...] = None*, ****instance*: *Any | None = None*, *row*: *Row[Unpack[TupleAny]] | RowMapping | None = None*, *identity_token*: *Any | None = None*) → *_IdentityKeyType[Any]*

Return an identity key.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

This is an alias of **identity_key()**.

property **identity_map**: *IdentityMap*

Proxy for the **Session.identity_map** attribute on behalf of the **AsyncSession** class.

method **sqlalchemy.ext.asyncio.AsyncSession.in_nested_transaction**() → *bool*

Return True if this **Session** has begun a nested transaction, e.g. SAVEPOINT.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

Added in version 1.4.

method **sqlalchemy.ext.asyncio.AsyncSession.in_transaction**() → *bool*

Return True if this **Session** has begun a transaction.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

Added in version 1.4.

See also

Session.is_active

property **info**: *Any*

A user-modifiable dictionary.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

The initial value of this dictionary can be populated using the `info` argument to the **Session** constructor or **sessionmaker** constructor or factory methods. The dictionary here is always local to this **Session** and can be modified independently of all other **Session** objects.

method **sqlalchemy.ext.asyncio.AsyncSession.async invalidate**() → *None*

Close this Session, using connection invalidation.

For a complete description, see **Session.invalidate()**.

property **is_active**: *Any*

True if this **Session** not in “partial rollback” state.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

Changed in version 1.4: The **Session** no longer begins a new transaction immediately, so this attribute will be False when the **Session** is first instantiated.

“partial rollback” state typically indicates that the flush process of the **Session** has failed, and that the **Session.rollback()** method must be emitted in order to fully roll back the transaction.

If this **Session** is not in a transaction at all, the **Session** will autobegin when it is first used, so in this case **Session.is_active** will return True.

Otherwise, if this **Session** is within a transaction, and that transaction has not been rolled back internally, the **Session.is_active** will also return True.

See also

“This Session’s transaction has been rolled back due to a previous exception during flush.” (or similar)

Session.in_transaction()

method **sqlalchemy.ext.asyncio.AsyncSession.is_modified**(instance: *object*, include_collections: bool = True) → bool

Return `True` if the given instance has locally modified attributes.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

This method retrieves the history for each instrumented attribute on the instance and performs a comparison of the current value to its previously flushed or committed value, if any.

It is in effect a more expensive and accurate version of checking for the given instance in the **Session.dirty** collection; a full test for each attribute’s net “dirty” status is performed.

E.g.:

```
return session.is_modified(someobject)
```

A few caveats to this method apply:

- Instances present in the **Session.dirty** collection may report `False` when tested with this method. This is because the object may have received change events via attribute mutation, thus placing it in **Session.dirty**, but ultimately the state is the same as that loaded from the database, resulting in no net change here.
- Scalar attributes may not have recorded the previously set value when a new value was applied, if the attribute was not loaded, or was expired, at the time the new value was received - in these cases, the attribute is assumed to have a change, even if there is ultimately no net change against its database value. SQLAlchemy in most cases does not need the “old” value when a set event occurs, so it skips the expense of a SQL call if the old value isn’t present, based on the assumption that an UPDATE of the scalar value is usually needed, and in those few cases where it isn’t, is less expensive on average than issuing a defensive SELECT.

The “old” value is fetched unconditionally upon set only if the attribute container has the **active_history** flag set to `True`. This flag is set typically for primary key attributes and scalar object references that are not a simple many-to-one. To set this flag for any arbitrary mapped column, use the **active_history** argument with **column_property()**.

Parameters:

- instance** – mapped instance to be tested for pending changes.
- include_collections** – Indicates if multivalued collections should be included in the operation. Setting this to `False` is a way to detect only local-column based properties (i.e. scalar columns or many-to-one foreign keys) that would result in an UPDATE for this instance upon flush.

method **sqlalchemy.ext.asyncio.AsyncSession.async merge**(instance: *_O*, *load: bool = True, options: *Sequence[ORMOption] | None = None*) → *_O*

Copy the state of a given instance into a corresponding instance within this **AsyncSession**.

See also

Session.merge() - main documentation for merge

method **sqlalchemy.ext.asyncio.AsyncSession.async merge_all**(instances: *Iterable[_O]*, *load: bool = True, options: *Sequence[ORMOption] | None = None*) → *Sequence[_O]*

Calls **AsyncSession.merge()** on multiple instances.

See also

Session.merge_all() - main documentation for merge_all

property **new: Any**

The set of all instances marked as ‘new’ within this **Session**.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

property **no_autoflush: Any**

Return a context manager that disables autoflush.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

e.g.:

```
with session.no_autoflush:

    some_object = SomeClass()
    session.add(some_object)
    # won't autoflush
    some_object.related_thing = session.query(SomeRelated).first()
```

Operations that proceed within the `with:` block will not be subject to flushes occurring upon query access. This is useful when initializing a series of objects which involve existing database queries, where the uncompleted object should not yet be flushed.

classmethod `sqlalchemy.ext.asyncio.AsyncSession.object_session(instance: object) → Session | None`

Return the **Session** to which an object belongs.

Proxied for the **Session** class on behalf of the **AsyncSession** class.

This is an alias of `object_session()`.

method `sqlalchemy.ext.asyncio.AsyncSession.async refresh(instance: object, attribute_names: Iterable[str] | None = None, with_for_update: ForUpdateParameter = None) → None`

Expire and refresh the attributes on the given instance.

A query will be issued to the database and all attributes will be refreshed with their current database value.

This is the async version of the `Session.refresh()` method. See that method for a complete description of all options.

See also

`Session.refresh()` - main documentation for refresh

method `sqlalchemy.ext.asyncio.AsyncSession.async reset() → None`

Close out the transactional resources and ORM objects used by this **Session**, resetting the session to its initial state.

Added in version 2.0.22.

See also

`Session.reset()` - main documentation for “reset”

Closing - detail on the semantics of `AsyncSession.close()` and `AsyncSession.reset()`.

method `sqlalchemy.ext.asyncio.AsyncSession.async rollback() → None`

Rollback the current transaction in progress.

See also

`Session.rollback()` - main documentation for “rollback”

method `sqlalchemy.ext.asyncio.AsyncSession.async run_sync(fn: ~typing.Callable[[_typing.Concatenate[_sqlalchemy.orm.session.Session, ~_P]], ~_sqlalchemy.ext.asyncio.session._T], *arg: ~typing._P, **kw: ~typing._P) → _T`

Invoke the given synchronous (i.e. not async) callable, passing a synchronous-style **Session** as the first argument.

This method allows traditional synchronous SQLAlchemy functions to run within the context of an asyncio application.

E.g.:

```
def some_business_method(session: Session, param: str) -> str:
    """A synchronous function that does not require awaiting

    :param session: a SQLAlchemy Session, used synchronously

    :return: an optional return value is supported

    """
    session.add(MyObject(param=param))
    session.flush()
    return "success"

async def do_something_async(async_engine: AsyncEngine) -> None:
    """an async function that uses awaiting"""
```

```
with AsyncSession(async_engine) as async_session:
    # run some_business_method() with a sync-style
    # Session, proxied into an awaitable
    return_code = await async_session.run_sync(
        some_business_method, param="param1"
    )
    print(return_code)
```

This method maintains the asyncio event loop all the way through to the database connection by running the given callable in a specially instrumented greenlet.

Tip

The provided callable is invoked inline within the asyncio event loop, and will block on traditional IO calls. IO within this callable should only call into SQLAlchemy's asyncio database APIs which will be properly adapted to the greenlet context.

See also

AsyncAttrs - a mixin for ORM mapped classes that provides a similar feature more succinctly on a per-attribute basis

AsyncConnection.run_sync()

Running Synchronous Methods and Functions under asyncio

method `sqlalchemy.ext.asyncio.AsyncSession.async scalar(statement: Executable, params: _CoreAnyExecuteParams / None = None, *, execution_options: OrmExecuteOptionsParameter = {}, bind_arguments: _BindArguments / None = None, **kw: Any) → Any`

Execute a statement and return a scalar result.

See also

Session.scalar() - main documentation for scalar

method `sqlalchemy.ext.asyncio.AsyncSession.async scalars(statement: Executable, params: _CoreAnyExecuteParams / None = None, *, execution_options: OrmExecuteOptionsParameter = {}, bind_arguments: _BindArguments / None = None, **kw: Any) → ScalarResult[Any]`

Execute a statement and return scalar results.

Returns:

a **ScalarResult** object

Added in version 1.4.24: Added **AsyncSession.scalars()**

Added in version 1.4.26: Added **async_scoped_session.scalars()**

See also

Session.scalars() - main documentation for scalars

AsyncSession.stream_scalars() - streaming version

method `sqlalchemy.ext.asyncio.AsyncSession.async stream(statement: Executable, params: _CoreAnyExecuteParams / None = None, *, execution_options: OrmExecuteOptionsParameter = {}, bind_arguments: _BindArguments / None = None, **kw: Any) → AsyncResult[Unpack[TupleAny]]`

Execute a statement and return a streaming **AsyncResult** object.

method `sqlalchemy.ext.asyncio.AsyncSession.async stream_scalars(statement: Executable, params: _CoreAnyExecuteParams / None = None, *, execution_options: OrmExecuteOptionsParameter = {}, bind_arguments: _BindArguments / None = None, **kw: Any) → AsyncScalarResult[Any]`

Execute a statement and return a stream of scalar results.

Returns:

an **AsyncScalarResult** object

Added in version 1.4.24.

See also

Session.scalars() - main documentation for scalars

AsyncSession.scalars() - non streaming version

attribute **sqlalchemy.ext.asyncio.AsyncSession.sync_session**: *Session*

Reference to the underlying **Session** this **AsyncSession** proxies requests towards.

This instance can be used as an event target.

See also

Using events with the asyncio extension

attribute **sqlalchemy.ext.asyncio.AsyncSession.sync_session_class**: *Type[Session]*

The class or callable that provides the underlying **Session** instance for a particular **AsyncSession**.

At the class level, this attribute is the default value for the **AsyncSession.sync_session_class** parameter. Custom subclasses of **AsyncSession** can override this.

At the instance level, this attribute indicates the current class or callable that was used to provide the **Session** instance for this **AsyncSession** instance.

Added in version 1.4.24.

class **sqlalchemy.ext.asyncio.AsyncSessionTransaction**

inherits from sqlalchemy.ext.asyncio.base.ReversibleProxy, sqlalchemy.ext.asyncio.base.StartableContext

A wrapper for the ORM **SessionTransaction** object.

This object is provided so that a transaction-holding object for the **AsyncSession.begin()** may be returned.

The object supports both explicit calls to **AsyncSessionTransaction.commit()** and **AsyncSessionTransaction.rollback()**, as well as use as an async context manager.

Added in version 1.4.

Member Name	Description
commit()	Commit this AsyncTransaction .
rollback()	Roll back this AsyncTransaction .

method **sqlalchemy.ext.asyncio.AsyncSessionTransaction.async commit()** → None

Commit this **AsyncTransaction**.

method **sqlalchemy.ext.asyncio.AsyncSessionTransaction.async rollback()** → None

Roll back this **AsyncTransaction**.

Previous: **ORM Extensions** Next: **Association Proxy**

© Copyright 2007-2026, the SQLAlchemy authors and contributors.

flambé! the dragon and **The Alchemist** image designs created and generously donated by **Rotem Yaari**.

Created using **Sphinx** 9.1.0. Documentation last generated: Fri 03 Apr 2026 03:31:14 PM EDT